

Sparse indicator quadratics: exact complexity and smoothed separator messages

Abstract

We minimize a positive-definite quadratic in which each continuous variable can be nonzero only if a penalized binary indicator is on. For sparse matrices, exact dynamic programming over a tree decomposition must pass functions, called messages, of the few continuous variables shared by subproblems. Each message is the minimum of one quadratic per choice of active indicators in the eliminated part, so few shared variables bound its dimension but not its number of quadratics. We show that perturbing each penalty by an independent uniform draw from a grid in $[-\sigma, \sigma]$ makes exact messages efficient to compute in expectation. For every draw, our algorithm solves the perturbed problem and computes every message exactly on a box of radius R , as a list containing every quadratic that attains the minimum somewhere on the box. At fixed treewidth, its expected bit complexity is polynomial in the input length and in C , H , μ^{-1} , R , and σ^{-1} , where C bounds the linear coefficients and the eigenvalues lie in $[\mu, H]$. The main tool lists, at finitely many points of the box, every near-optimal choice of active indicators, using only an approximate solver in which some indicators can be fixed. Two lower bounds explain why the running time depends on numerical values and why the penalties are perturbed. Exact optimization is NP-hard at bandwidth two even near the identity, so unless $\text{NP} \subseteq \text{ZPP}$ the polynomial dependence on C cannot be replaced by dependence on encoding length, even for fixed H , μ , and σ . Without perturbation, messages can need exponentially many quadratics even with local coefficients from a fixed finite set. The polynomial upper bound has high degree; we discuss its practical relevance.

1 Introduction

Many estimation problems ask for a vector that fits the data well and has few nonzero entries. A common model gives each continuous variable x_i a binary indicator z_i : the variable may be nonzero only if $z_i = 1$, and switching the indicator on costs a penalty λ_i . With a quadratic measure of fit, this gives the *indicator quadratic problem*

$$\min\{x^T Q x + c^T x + \lambda^T z : x_i(1 - z_i) = 0 \ (i \in [n]), \ z \in \{0, 1\}^n\},$$

where Q is a symmetric positive-definite matrix. We call a coordinate with $z_i = 1$ *active* and the set of active coordinates the *support* of a solution; an active coordinate may still be zero. Once the support is fixed, the best x solves a linear system, so the difficulty lies in choosing the support. Problems of this form arise in sparse regression, outlier detection in time series, inference in Gaussian hidden Markov models, and real-time monitoring of data streams [5, 6, 18].

Problems of this kind are NP-hard in general (see [11, 30] and the references there). Much work therefore develops strong convex formulations, often by describing the closed convex hull of the epigraph of the objective over the feasible set in an extended space [2, 11, 24, 27, 30]. The same line of work has identified structured cases that can be solved in polynomial time. These include Stieltjes matrices, whose off-diagonal entries are nonpositive, with linear coefficients of one sign

[2, 27]; matrices whose sparsity pattern forms a path [26] or, more generally, a tree [6]; matrices with a tridiagonal inverse, which arise in multi-period models [24]; and, through decision diagrams, quadratics of fixed rank and matrices whose sparsity pattern, or that of their inverse, is a tree with a fixed number of leaves [11]. Decision diagrams also give an additive approximation scheme for banded matrices of fixed bandwidth and condition number [18]. For sparse or inverse-sparse matrices, under conditions on the growth of graph neighborhoods, Choi et al. obtain approximate diagrams whose size is linear in n for each fixed accuracy [11].

This paper concerns sparse matrices whose *support graph*, with an edge ij whenever $Q_{ij} \neq 0$, has small treewidth. Informally, such a graph can be assembled from small, overlapping groups of vertices arranged along a tree, and removing the few vertices shared by two neighboring groups disconnects the vertices on one side of the tree of groups from those on the other. Such an arrangement is called a *tree decomposition*. Trees have treewidth one, and a matrix of bandwidth β , with $Q_{ij} = 0$ whenever $|i - j| > \beta$, has treewidth at most β ; Section 2 gives the precise definitions. For purely discrete problems on such graphs, the standard exact method is dynamic programming, which eliminates variables from the leaves of the tree of groups toward a chosen root. Because the eliminated part interacts with the rest only through a few shared variables, it suffices to record its best cost for each combination of their values. With few shared discrete variables, this table is small.

Indicator quadratics behave differently because the shared variables include continuous ones. The table then becomes a function: for each value t of the shared continuous variables (the *boundary values*), it gives the least cost of the eliminated part with these variables fixed at t . We call such a function a *message*, and an exact dynamic program needs every message exactly. A message is also useful in its own right: it gives the least cost and an optimal completion of the eliminated part for every boundary value at once, so it can be stored and reused when the rest of the problem changes. Bhatena et al. use this to update solutions online as new observations arrive [6].

Fixing which eliminated coordinates are active leaves a strictly convex quadratic in the active eliminated variables, and minimizing over them gives a quadratic function of t . A message is therefore the pointwise minimum of 2^m quadratics, one for each support of the m eliminated variables. Different supports can be optimal at different values of t : even a single eliminated variable with a positive penalty should be active only for those t that push its best value far enough from zero to pay for the penalty. A few shared variables thus keep a message low-dimensional, but they do not limit the number of quadratics needed to describe it.

On trees, a message depends on one variable, and Bhatena et al. showed that linearly many quadratics suffice [6]. Beyond trees, they gave an exact algorithm whose efficiency requires further conditions on the growth of graph neighborhoods and on how many supports of each local subproblem are nearly optimal, and they stated that whether these functions have a compact representation is an open question [5, Section 2.2]. For representations as minima of quadratics, we show that the answer is negative already at treewidth two, but that a small random perturbation of the penalties makes the expected number of quadratics needed polynomial at each fixed treewidth.

1.1 Results

Fix $\sigma > 0$ and replace each penalty λ_i by $\lambda_i + \xi_i$, where the *perturbations* ξ_i are independent uniform draws from a grid of at least $2n$ equally spaced values in $[-\sigma, \sigma]$. Only the penalties are perturbed. This is the setting of smoothed analysis, as used for binary optimization [3, 29]: an adversary chooses Q , c , and λ , and expectations are taken over the small random perturbation alone.

Our main result, Theorem 5.2, gives an algorithm that, for every outcome of the perturbation, solves the perturbed problem exactly and computes every message of a given tree decomposition exactly on a box of boundary values with a given radius R . Each message is returned as a *dictionary*:

a list of quadratics whose pointwise minimum equals the message on the box. The dictionary contains the quadratic of every support that is *optimal somewhere* on the box, that is, optimal for at least one value t in the box. For each fixed treewidth, the expected number of bit operations is polynomial in the input length and in the values, not the bit lengths, of C , H , μ^{-1} , R , and σ^{-1} , where $|c_i| \leq C$ for all i and the eigenvalues of Q lie in $[\mu, H]$. The degree of this polynomial depends on the treewidth.

At a fixed boundary value, the random penalties make near ties unlikely: the expected number of supports whose cost is within $\sigma/(2n)$ of the optimum is at most Euler's number $e \approx 2.72$. At every point of a fine finite net of boundary values that covers the box, the algorithm lists every support within $\sigma/(6n)$ of the optimum, and only supports within $\sigma/(2n)$, so each list has expected length at most e . For this it needs only an approximate solver that accepts fixed indicators; for indicator quadratics, a dynamic program over rounded continuous values provides one. The quadratics are Lipschitz continuous on the box, so every support that is optimal somewhere on the box is nearly optimal at a nearby net point and is listed there; the union of the lists is therefore a dictionary. The algorithm computes each message directly in this way, from its own subproblem, instead of passing messages up the tree. The number of net points and the size of the tables of the rounded dynamic program are polynomial in n , C , H , μ^{-1} , R , and σ^{-1} , with the treewidth in the exponent; this is where these values and the treewidth enter the running time (Section 2.5).

Theorem 4.3 proves this enumeration step for any family of supports whose costs are Lipschitz in a parameter and are perturbed by random penalties. Its ingredients are classical (Section 3); to the best of our knowledge, what is new is the completeness guarantee for approximate solvers and its use to cover a whole parameter domain.

The answers are exact for the perturbed instance, not for the original one. Any exact solution of the perturbed problem gives lower and upper bounds on the original optimal value that differ by at most $n\sigma$ (Section 5.4). If only an approximate optimum of the original problem is wanted, the dynamic program over rounded values gives one directly; the point of Theorem 5.2 is exact messages. A dictionary may contain supports that are never optimal, and the algorithm does not compute the part of the box on which each support is optimal.

Two lower bounds explain why the running time depends on numerical values and why near ties must be controlled, whether by the perturbation or by conditions such as those of Bhathena et al. [5]. Theorem 6.1 shows that the problem is NP-hard even when Q has bandwidth two and is arbitrarily close to the identity matrix, so small treewidth and good conditioning alone do not give a polynomial exact algorithm unless $P = NP$. By scaling the linear coefficients and penalties of these instances, Corollary 6.2 carries this over to the perturbed problem with a fixed family of matrices, hence fixed H and μ , and a fixed σ . Unless $NP \subseteq ZPP$, the polynomial dependence of Theorem 5.2 on C therefore cannot be replaced by a dependence on the encoding length. Here ZPP is the class of problems solved by randomized algorithms that never err and run in expected polynomial time; it appears because the running time of Theorem 5.2 is bounded only in expectation.

Theorem 7.2 shows that messages can be large even when optimization is easy. Its instances have $2n$ variables, treewidth two, unit penalties, a matrix close to the identity, and local coefficients from a fixed finite set. Their global optimum is trivial to find, and 2^n different supports attain it, but each of these supports is the unique best one for boundary values in a short interval of its own. Hence one message, a function of a single variable, needs at least 2^n distinct formulas in any exact representation as a minimum of polynomials or as a piecewise polynomial function. With perturbed penalties, Theorem 5.2 applies to the same instances and makes the expected dictionary size polynomial. If this message is needed only within a uniform error ε , a number of its quadratics bounded independently of n suffices, but this number grows as a power of $1/\varepsilon$, with matching upper and lower exponents (Theorem 7.3).

Appendix A concerns representations that depend only on Q and can therefore be built once and reused for all linear coefficients and penalties. It studies the inverse-principal polytope, which Wei et al. use to describe the closed convex hull of the epigraph of $x^T Q x$ with indicators [30]. On a well-conditioned star, where optimization is easy [6], every exact extended formulation (lift) of this polytope by linear, second-order-cone, or semidefinite constraints with blocks of bounded order has exponential size (Theorem A.1 and Corollary A.2).

The main result is a complexity result, and its polynomial bound has high degree. We do not propose the algorithm as a practical solver and report no computational experiments; Section 8 discusses which parts of the analysis may still be useful in practice.

Section 2 explains dynamic programming over tree decompositions for this problem, defines messages and dictionaries, describes the perturbation, and outlines the proof of the main result. Section 3 compares our results with prior work. Section 4 proves the general counting and enumeration results, and Section 5 applies them to prove Theorem 5.2 and to bound the original optimum. Sections 6 and 7 prove the two lower bounds, Section 8 discusses practical relevance and open questions, and Appendix A contains the results on stars.

2 Preliminaries

Our algorithm computes the value functions that dynamic programming passes between parts of a sparse problem. We first state the problem and show why it is easy once the support is fixed. We then run dynamic programming by hand on a chain of variables, where the basic difficulty already appears: the eliminated part must be summarized by a function of a continuous variable, and its optimal support changes with that variable. Tree decompositions extend this to general sparse matrices, where the number of such supports can become exponential. Sections 2.4 and 2.5 define the messages we compute, describe the perturbation, and outline the proof of the main result.

2.1 The problem

For $n \geq 1$, the *indicator quadratic problem* is

$$v_\xi = \min \left\{ x^T Q x + c^T x + (\lambda + \xi)^T z : x \in \mathbb{R}^n, z \in \{0, 1\}^n, x_i(1 - z_i) = 0 \ (i \in [n]) \right\}. \quad (1)$$

Here $[n] = \{1, \dots, n\}$, $Q \in \mathbb{Q}^{n \times n}$ is symmetric, and $c, \lambda \in \mathbb{Q}^n$. Each continuous variable x_i has a binary indicator z_i . The constraint $x_i(1 - z_i) = 0$ allows $x_i \neq 0$ only if $z_i = 1$, and the choice $z_i = 1$ costs the penalty $\lambda_i + \xi_i$. The penalties λ_i may have either sign. The vector $\xi \in \mathbb{Q}^n$ is the *perturbation* of the penalties, and $\xi = 0$ gives the original problem. Section 2.5 describes how ξ is drawn at random, and the exact results of Section 5 concern the perturbed problem. We are given rational bounds

$$0 < \mu I \preceq Q \preceq H I, \quad |c_i| \leq C \quad (i \in [n]). \quad (2)$$

Thus Q is positive definite with all eigenvalues in $[\mu, H]$. These numbers enter the running-time bound of Theorem 5.2. All norms are Euclidean unless stated otherwise. For index sets $A, B \subseteq [n]$, we write Q_{AB} for the submatrix of Q with rows in A and columns in B , and x_A for the subvector of x indexed by A .

The *support* of a feasible pair (x, z) is $A = \{i : z_i = 1\}$, and a coordinate i is *active* if $z_i = 1$. The constraint $x_i(1 - z_i) = 0$ forces $x_i = 0$ only for inactive coordinates. An active coordinate may therefore be zero, and the support need not equal $\{i : x_i \neq 0\}$. For instance, if $\lambda_i + \xi_i < 0$, activating i lowers the cost even if x_i stays zero.

Once the support is fixed, the problem is easy. For a support A , the inactive coordinates are zero, and what remains is the minimization of $x_A^T Q_{AA} x_A + c_A^T x_A$ over $x_A \in \mathbb{R}^A$. The matrix Q_{AA} is a principal submatrix of Q , hence positive definite, so this quadratic is strictly convex. Setting its gradient $2Q_{AA}x_A + c_A$ to zero gives a linear system, whose solution and optimal value are

$$x_A = -\frac{1}{2}Q_{AA}^{-1}c_A, \quad x_A^T Q_{AA} x_A + c_A^T x_A = -\frac{1}{4}c_A^T Q_{AA}^{-1}c_A.$$

The optimum v_ξ is therefore the least of the 2^n numbers $-\frac{1}{4}c_A^T Q_{AA}^{-1}c_A + \sum_{i \in A}(\lambda_i + \xi_i)$, one for each support $A \subseteq [n]$, where the empty support has value zero. All the difficulty lies in choosing the support.

When Q is sparse, this choice can be made one group of variables at a time. The sparsity pattern is described by the *support graph* of Q , which has vertex set $[n]$ and an edge ij for each pair $i \neq j$ with $Q_{ij} \neq 0$. Two variables are adjacent exactly when the objective contains their product.

2.2 Dynamic programming on a chain

Suppose first that Q is tridiagonal, that is, $Q_{ij} = 0$ whenever $|i - j| > 1$, so that the support graph is contained in the path $1, 2, \dots, n$. Such matrices arise, for example, when x_i is the value of a signal at time i and the objective couples consecutive times. Write $\rho_i = \lambda_i + \xi_i$. The objective of (1) is then

$$\sum_{i=1}^n (Q_{ii}x_i^2 + c_i x_i + \rho_i z_i) + \sum_{i=1}^{n-1} 2Q_{i,i+1}x_i x_{i+1}, \quad (3)$$

so each pair (x_i, z_i) interacts only with its neighbors in the chain.

Eliminate (x_1, z_1) first. The terms that contain x_1 or z_1 are $Q_{11}x_1^2 + c_1 x_1 + \rho_1 z_1 + 2Q_{12}x_1 x_2$, and they involve the other variables only through x_2 . Fix $x_2 = t$ and minimize over both choices of z_1 . If $z_1 = 0$, then $x_1 = 0$ and these terms vanish. If $z_1 = 1$, they form a strictly convex quadratic in x_1 , which is minimized at $x_1 = -(c_1 + 2Q_{12}t)/(2Q_{11})$. The least cost of the eliminated pair, as a function of t , is therefore

$$M_1(t) = \min \left\{ 0, \rho_1 - \frac{(c_1 + 2Q_{12}t)^2}{4Q_{11}} \right\}. \quad (4)$$

The constant 0 belongs to the empty support and the second quadratic to the support $\{1\}$. Which of the two is smaller depends on t . Suppose that $\rho_1 > 0$ and $Q_{12} \neq 0$. The empty support is optimal where $|c_1 + 2Q_{12}t| \leq 2\sqrt{Q_{11}\rho_1}$, and $\{1\}$ is optimal where the reverse inequality holds. Both are optimal at the two points where equality holds, which are irrational in general. Thus, already in this smallest case, the optimal support changes with t on every interval $[-R, R]$ with R large enough.

This is the essential difference from a discrete dynamic program. In a shortest-path or knapsack program, the state passed from one stage to the next takes finitely many values, and the recursion stores one number per state. Here the state is the value t of the continuous variable x_2 , which is decided only later, when the remaining variables are optimized. The recursion must therefore keep the whole function M_1 ; neither one number nor one best support describes the eliminated part.

Next eliminate (x_2, z_2) with $x_3 = t$ fixed. The cost of (x_1, z_1) is now summarized by $M_1(x_2)$, and the other terms that involve x_2 or z_2 are $Q_{22}x_2^2 + c_2 x_2 + \rho_2 z_2 + 2Q_{23}x_2 t$. Hence

$$M_2(t) = \min_{\substack{x_2 \in \mathbb{R}, z_2 \in \{0,1\} \\ x_2(1-z_2)=0}} \{M_1(x_2) + Q_{22}x_2^2 + c_2 x_2 + \rho_2 z_2 + 2Q_{23}x_2 t\}.$$

Choosing one of the two terms of the minimum (4) and a value of z_2 fixes a support $A \subseteq \{1, 2\}$. Once A is fixed, what remains is the minimization of a strictly convex quadratic in the active

variables among x_1, x_2 , whose linear term depends affinely on t , so its value is a quadratic function of t . Hence M_2 is the minimum of four quadratics in t , one for each support of $\{1, 2\}$.

Continuing in this way, suppose that $(x_1, z_1), \dots, (x_k, z_k)$ have been eliminated, where $k < n$. Their least cost depends on the remaining variables only through $t = x_{k+1}$, and

$$M_k(t) = \min_{A \subseteq \{1, \dots, k\}} F_A(t), \quad (5)$$

where $F_A(t)$ is the least total of the terms that contain an eliminated variable, given that the eliminated variables have support A and $x_{k+1} = t$. By the closed form of Section 2.1, with c_A replaced by $c_A + 2Q_{A, \{k+1\}}t$, each F_A is a quadratic function of t ; it is given in general by (9) below. Thus M_k is the minimum of up to 2^k quadratics, one for each support of the eliminated variables, and each step computes M_k from M_{k-1} in the same way as M_2 from M_1 . After $n - 1$ steps only (x_n, z_n) remains, and minimizing $M_{n-1}(x_n) + Q_{nn}x_n^2 + c_nx_n + \rho_nz_n$ over it gives v_ξ . If a support A attains M_{n-1} at an optimal choice of x_n , then the minimizer with support A gives optimal values of $x_1, \dots, x_{n-1}$.

Storing all 2^k quadratics is out of the question for large k , and on a chain it is also unnecessary. Liu et al. solve the chain in quadratic time as a shortest-path problem whose arcs are blocks of consecutive active coordinates [26]. This works because an inactive coordinate is zero and splits the chain into independent blocks, so no continuous value has to be passed across it. Bhatena et al. show that on a chain, and more generally when the support graph of Q is a tree, these functions of one variable split into linearly many intervals, each with a single quadratic piece; on a chain, each elimination step adds at most two pieces [6, Proposition 1 and Lemmas 3 and 4]. Their algorithm stores only these pieces, never forms the 2^k quadratics, and solves the problem on trees in quadratic time [6].

As noted in the Introduction, M_k can be stored and reused when the rest of the problem changes. If a variable x_{n+1} is appended to the chain, for example, only one more elimination step is needed, starting from the stored M_{n-1} .

2.3 Tree decompositions and separators

On a chain, every block of eliminated variables touches the rest of the problem through a single variable. For a general sparse matrix, eliminating a set of variables leaves a function of all remaining variables adjacent to that set, and the order of elimination decides how many there are. Tree decompositions organize the elimination so that this number stays small.

A *tree decomposition* of the support graph is a tree whose nodes u carry vertex sets $B_u \subseteq [n]$, called *bags*, such that every vertex lies in some bag, both endpoints of every edge lie in a common bag, and for each vertex the nodes whose bags contain it form a connected subtree. We call the last condition the *running-intersection property*. The *width* of the decomposition is $\max_u |B_u| - 1$, and the *treewidth* of the graph is the least width of a tree decomposition. A tree with at least one edge has treewidth one, with one bag for each edge, so treewidth measures how far a graph is from a tree.

We assume that a decomposition of width w with b bags is supplied. For fixed w , such a decomposition can be computed in linear time when one exists [9]. For example, if Q has bandwidth β , that is, $Q_{ij} = 0$ whenever $|i - j| > \beta$, then the bags $\{i, \dots, \min\{i + \beta, n\}\}$ for $i = 1, \dots, n$, placed along a path, form a tree decomposition of width at most β . A tree decomposition whose tree is a path is a *path decomposition*, and the least width of a path decomposition is the *pathwidth*. The chain of Section 2.2 is the case $\beta = 1$. Figure 1 shows the case $\beta = 2$ and $n = 6$, with the bags $\{5, 6\}$ and $\{6\}$ dropped because they lie inside $\{4, 5, 6\}$. Tree decompositions are a standard tool for exploiting sparsity in optimization. For example, Bienstock and Muñoz use them to build

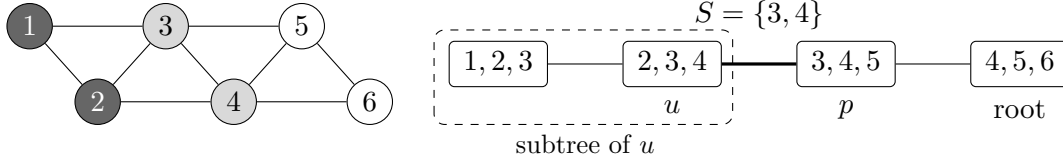


Figure 1: Left: the support graph of a 6×6 matrix of bandwidth two. Right: a tree decomposition of width two whose bags $\{i, i+1, i+2\}$, $i = 1, \dots, 4$, are the triangles of the graph, placed along a path and rooted at $\{4, 5, 6\}$. For the node u with parent p , the separator is $S = B_u \cap B_p = \{3, 4\}$ (light) and the internal set is $I = \{1, 2\}$ (dark). The variables x_1, x_2 interact with x_5, x_6 only through x_3, x_4 .

polynomial-size linear programs that approximate mixed-integer polynomial optimization problems whose constraint intersection graph has bounded treewidth [8].

The edges of a rooted decomposition give the separators. Root the supplied decomposition at any node. For a non-root node u with parent p , let $S = B_u \cap B_p$, and let I be the set of vertices that lie in a bag of the subtree rooted at u but not in S . We call S the *separator* and I the *internal set* of u . No edge of the support graph joins I to $[n] \setminus (I \cup S)$, so the variables in I interact with the rest of the problem only through those in S . Indeed, suppose ij is such an edge with $i \in I$, and take a bag containing both endpoints. If the bag lies in the subtree of u , then $j \in I \cup S$. Otherwise i lies in bags on both sides of the tree edge up , so the running-intersection property gives $i \in B_u \cap B_p = S$. Both cases contradict the choice of ij . For the chain, with the bags $\{i, \min\{i+1, n\}\}$ rooted at the last bag $\{n\}$, the node with bag $\{k, k+1\}$ has separator $\{k+1\}$ and internal set $\{1, \dots, k\}$, which is exactly the situation after k elimination steps.

2.4 Messages, branches, and dictionaries

The value functions M_k of the chain generalize as follows. Let $I, S \subseteq [n]$ be disjoint; the main case is the internal set and the separator of a node of the decomposition (Section 2.3). Write t for the boundary values x_S , and fix a box radius $R \geq 0$. The *message* is the conditional value function

$$M_{I,S}(t) = \min_{\substack{x_I \in \mathbb{R}^I, z_I \in \{0,1\}^I \\ x_i(1-z_i)=0 \ (i \in I)}} \left\{ x_I^T Q_{II} x_I + (c_I + 2Q_{IS}t)^T x_I + \sum_{i \in I} (\lambda_i + \xi_i) z_i \right\}, \quad t \in [-R, R]^{|S|}. \quad (6)$$

The message keeps exactly the objective terms that involve an internal variable: the internal quadratic, the coupling $2t^T Q_{SI} x_I$, and the internal linear terms and penalties. It leaves out the boundary unary terms $Q_{ss} t_s^2 + c_s t_s$ ($s \in S$), the edges within S , and the boundary penalties. For fixed boundary variables (x_S, z_S) , these terms add the same amount to every internal solution, so they do not change which internal support is optimal. They are counted once, with the rest of the problem. On the box, the function M_k of the chain is the message with $I = \{1, \dots, k\}$ and $S = \{k+1\}$. The radius R is part of the input, and the running time of Section 5 grows polynomially with R at fixed width. The radius $R = nC/(2\mu)$ suffices for optimization: for every ξ , this box contains the boundary values x_S of every optimal solution (Section 5.1).

Now let S and I be the separator and internal set of a non-root node u of the rooted decomposition. Fix all variables outside I , with $x_S = t \in [-R, R]^{|S|}$. Since no edge joins I to a vertex outside $I \cup S$, the objective of (1) equals the objective in (6) plus terms that do not involve (x_I, z_I) . Minimizing over (x_I, z_I) therefore gives $M_{I,S}(t)$ plus those terms. Thus $M_{I,S}$ is the exact subtree message: it is all that the rest of the problem needs to know about the subtree. We also include the root message

$M_{[n],\emptyset}$. Its box is a single point, and its value is the global optimum v_ξ . In total there are at most b messages, each with $|S| \leq w + 1$.

Boundary indicators need no separate messages. The internal problem depends on the boundary only through $t = x_S$. A boundary indicator z_s only decides whether t_s may be nonzero and adds the penalty $\lambda_s + \xi_s$, which lies outside the message. For each assignment of the boundary indicators, the contribution of the subtree is therefore $M_{I,S}$ restricted to the slice of the box where $t_s = 0$ whenever $z_s = 0$. One dictionary serves all $2^{|S|}$ assignments.

For a fixed support $A \subseteq I$, the coordinates of I outside A are zero, and the closed form of Section 2.1, with c_A replaced by $c_A + 2Q_{AS}t$, gives the minimizer and the value:

$$x_A(t) = -\frac{1}{2}Q_{AA}^{-1}(c_A + 2Q_{AS}t), \quad (7)$$

$$q_A(t) = -\frac{1}{4}(c_A + 2Q_{AS}t)^T Q_{AA}^{-1}(c_A + 2Q_{AS}t) + \sum_{i \in A} \lambda_i, \quad (8)$$

$$F_A(t) = q_A(t) + \sum_{i \in A} \xi_i, \quad (9)$$

$$M_{I,S}(t) = \min_{A \subseteq I} F_A(t). \quad (10)$$

We call q_A and F_A the unperturbed and perturbed *branches* of A ; they agree when $\xi = 0$. The word refers to one term of the minimum (10), not to branching in branch-and-bound. The empty support has $q_\emptyset = F_\emptyset = 0$. Each branch is a concave quadratic in t , because Q_{AA}^{-1} is positive definite. It is a *full* quadratic: one polynomial formula on all of $\mathbb{R}^{|S|}$, not only on the region where A is optimal. Its coefficients are rational when ξ is rational. In the first step of the chain, (4) lists the two branches $F_\emptyset = 0$ and $F_{\{1\}}$ of the message with $I = \{1\}$ and $S = \{2\}$.

By (10), a message is the pointwise minimum of $2^{|I|}$ branches, but only some of them are needed. A *dictionary* for $M_{I,S}$ is a set of perturbed branches F_A , each stored with its support A , whose pointwise minimum equals $M_{I,S}$ on $[-R, R]^{|S|}$. A support A is *optimal somewhere* on the box if $F_A(t) = M_{I,S}(t)$ for at least one t in the box. Branches whose supports are never optimal on the box can be dropped without changing the message there.

A dictionary may still contain supports that are never optimal on the box, and it does not describe the region where each branch is optimal. These regions can end at irrational points, as already in (4). Everything a dictionary stores is rational. Hence, at a rational t in the box, the least branch value gives $M_{I,S}(t)$ exactly, and (7) for a minimizing support gives an optimal internal solution. The dictionaries of Section 5 contain every support optimal somewhere, including supports optimal only at a single tie point, so at every boundary value in the box they give all optimal internal supports, not just one.

Exact dynamic programming computes the messages from the leaves of the rooted decomposition toward the root, as on the chain. Let u be a node with separator S and internal set I (for the root, take $S = \emptyset$ and $I = [n]$). Let $v_1, \dots, v_r$ be its children, with separators S_j and internal sets I_j , and let $I_0 = B_u \setminus S$ be the variables eliminated at u . Then I is the disjoint union of $I_0, I_1, \dots, I_r$ by the running-intersection property: a vertex of B_u that lies in a bag in the subtree of v_j also lies in B_{v_j} , hence in $S_j = B_u \cap B_{v_j}$, so no I_j meets B_u ; a vertex in bags in the subtrees of two different children lies in B_u , so it lies in no I_j ; and every vertex of I outside B_u lies in some I_j . Each S_j lies in $B_u = I_0 \cup S$, so x_{S_j} is determined by x_{I_0} and $t = x_S$. Splitting the terms of (6) accordingly gives

$$M_{I,S}(t) = \min_{\substack{x_{I_0} \in \mathbb{R}^{I_0}, z_{I_0} \in \{0,1\}^{I_0} \\ x_i(1-z_i)=0 \ (i \in I_0)}} \left\{ \Phi_u(x_{I_0}, z_{I_0}, t) + \sum_{j=1}^r M_{I_j, S_j}(x_{S_j}) \right\},$$

where $\Phi_u(x_{I_0}, z_{I_0}, t)$ is the objective of (6) with I replaced by I_0 . Every term that involves a variable of I_j , including its coupling with x_{S_j} , belongs to the child message M_{I_j, S_j} . Each child message is given by the minimum in (6) at every $x_{S_j} \in \mathbb{R}^{|S_j|}$, not only on its box, because x_{S_j} may include coordinates of x_{I_0} , which range over all of $\mathbb{R}$. The chain step from M_{k-1} to M_k is the case $r = 1$ and $I_0 = \{k\}$. Fixing a support $A_0 \subseteq I_0$ and one branch F_{A_j} of each child fixes the internal support $A = A_0 \cup A_1 \cup \dots \cup A_r$. Minimizing $\Phi_u + \sum_j F_{A_j}(x_{S_j})$ over x_{I_0} with support A_0 then gives $F_A(t)$, because each F_{A_j} already minimizes over the variables of A_j . Thus, if each child message is written, on all of $\mathbb{R}^{|S_j|}$, as a minimum of d_j branches, the message of u is a minimum of up to $2^{|I_0|} d_1 \dots d_r$ candidate branches. The exact algorithm of Bhatena et al. forms these candidates and then discards those that it identifies as never optimal on a box [5, Sections 4 and 5].

The difficulty is the number of branches. A separator with k variables makes a message a function of k variables, but it does not bound the number of branches needed to represent it, even after pruning: Section 7 gives, at width two, a message of one boundary variable that needs exponentially many branches. The cause is near ties among many supports. Bhatena et al. limit near ties by conditions on the instance (Section 3); our main result instead controls them by perturbing the penalties at random.

2.5 The perturbation and an outline of the proof

Theorem 5.2, stated in Section 5.3, makes the main result of Section 1 precise. Each penalty λ_i is replaced by $\lambda_i + \xi_i$, where $\xi_1, \dots, \xi_n$ are independent and uniform on the *perturbation grid*

$$\left\{ -\sigma + \frac{2\sigma j}{N-1} : j = 0, \dots, N-1 \right\} \quad (11)$$

of N equally spaced points in $[-\sigma, \sigma]$, including both endpoints. Here $\sigma > 0$ is rational and N is the smallest power of two with $N \geq 2n$. Each ξ_i therefore uses $\lceil \log_2(2n) \rceil$ random bits, and the perturbed penalties stay rational. The algorithm uses ξ only through $\lambda + \xi$, so it may draw ξ itself or receive an already perturbed instance. Running times are measured by bit complexity, which counts bit operations and so accounts for the sizes of all rational numbers that the algorithm forms.

The proof has four ingredients. The first is a counting argument at one fixed boundary value t . The perturbation adds $\sum_{i \in A} \xi_i$ to the cost of each support A . Fix all coordinates of ξ except ξ_i . If the best support that contains i and the best one that does not are both within a of the optimum, then ξ_i lies in a fixed interval of length $2a$, which is unlikely when a is small compared with σ . Many near-optimal supports force this event for many coordinates at once. Lemma 4.1 makes this precise and shows that the expected number of supports within $\sigma/(2n)$ of the optimum at t is at most $e \approx 2.72$.

The second ingredient lists these supports. It needs only approximate optimization, which matters because the solver we use for indicator quadratics rounds the continuous variables to a finite grid. A *restricted additive oracle* is an algorithm that, with some indicators fixed, returns a support consistent with them together with its exact cost, which is within a given ε of the best such cost. Lawler's partition method splits the supports into cells by fixing indicator bits, much like the subproblems of branch-and-bound, and queries the oracle on each cell. Because each reported cost is exact and within ε of the best cost in its cell, the search can certify when no near-optimal support remains. This certified version of the method lists every support within $\sigma/(6n)$ of the optimum and only supports within $\sigma/(2n)$ (Lemma 4.2), so its expected output size is at most e .

The third ingredient passes from one boundary value to the whole box. A *parameter net* is a finite set of points such that every point of the box lies within a given distance of one of them. The branches are Lipschitz on the box, so a support that is optimal at some t is near-optimal at a

nearby net point. Running the enumeration at every point of the net therefore finds every support optimal somewhere on the box (Theorem 4.3). Section 4 proves these three ingredients for general families of supports, without quadratics or graphs.

The fourth ingredient is the oracle itself. For indicator quadratics, a dynamic program over a grid of continuous values on the tree decomposition provides it (Lemma 5.1), and Section 5 combines it with the first three ingredients. Each message is computed separately, without the exact recursion of Section 2.4: every oracle call for $M_{I,S}$ runs this grid dynamic program over all of I , so messages whose subproblems overlap repeat work. The spectral bounds (2) control the Lipschitz constant of the branches and the grid resolution, and the box must be bounded for the net to be finite. The width enters as the exponent $w + 1$ of the number of net points and of the size of each grid table. The running-time bound of Theorem 5.2 is therefore polynomial for each fixed w , but its exponent depends on w . It does not show fixed-parameter tractability in w , which would require a bound $f(w)p$ with a polynomial p whose degree does not depend on w .

3 Related work

Our results draw mainly on two bodies of work. Research on indicator quadratics with sparse matrices supplies the model, the messages of Section 2, and the structures under which exact or approximate methods are known to be efficient. Smoothed analysis of discrete optimization supplies the counting and scaling arguments behind Theorem 5.2 and Corollary 6.2. Sections 6–8 give further detail next to individual results.

3.1 Exact algorithms and polynomial cases

For tridiagonal matrices, whose support graph is a path, Liu, Fattahi, Gómez, and Küçükyavuz solve the problem in quadratic time by dynamic programming and give a compact extended formulation of the closed convex hull of the epigraph [26]. Bhathena, Fattahi, Gómez, and Küçükyavuz [6] solve the problem on arbitrary trees, also in quadratic arithmetic time and memory. Their dynamic program passes parametric costs and their conjugates, and our messages play the role of their parametric costs. Theorem 5.2 does not improve this tree-specific guarantee.

For graphs of larger treewidth, the same authors build exact parametric dynamic programs over a tree decomposition [5]. Their local parametric costs, defined for a whole bag, are minima of one quadratic per support, like our messages [5, Section 2.2 and Lemma 1]. They call these quadratics quadratic pieces; we call them branches. Their costs also include the quadratic and linear terms of the bag variables, so their quadratics are strongly convex rather than concave; these terms are the same for every internal support and do not change which supports are optimal. Their dynamic program computes these costs bottom-up, as outlined in Section 2.4, and after each step discards the branches that a pairwise test shows are not optimal anywhere on a box of boundary values that contains the optimal solutions [5, Definition 3 and Section 5].

The running-time bound of Bhathena et al. depends on the treewidth, the conditioning of Q , the volume growth of the support graph, a deterministic margin (k, η) , and the radius U of the box on which they prune, which they take proportional to $\|c\|_\infty$ [5, Lemma 3, Definition 5, Lemma 9, and Theorem 1]. Volume growth measures how fast the number of vertices within a given graph distance of a bag grows with that distance, and the margin bounds by k the number of η -optimal local supports. When all of these are fixed, the bound is linear in n . With linear volume growth it is polynomial in U^2/η , so, like Theorem 5.2, it grows with the size of the linear coefficients [5, Theorem 1 and Corollary 1].

Their pruning compares only supports that agree on the vertices within a certain graph distance of a bag and keeps at most k of them for each such pattern; volume growth bounds the number of these vertices, and hence of the patterns [5, Lemmas 8 and 9]. Theorem 5.2 needs neither the margin nor volume growth: under penalty perturbation, Lemma 4.1 bounds the expected number of near-optimal supports of a whole subproblem at each net point. On the other hand, Theorem 5.2 concerns the perturbed problem, and its bound is a polynomial of high degree. Neither result implies the other. Section 8 estimates the size of our bound, states their margin condition precisely, and explains how the perturbation affects it.

Other polynomial cases rest on special matrix structure. Problems with a Stieltjes matrix, whose off-diagonal entries are nonpositive, and with linear coefficients of one sign can be solved in polynomial time [2], as recalled in [27]. Lee, Gómez, and Atamtürk study multi-period problems in which the states follow exact affine dynamics, the quadratic part of the cost involves only the states, and only the controls carry indicators. Projecting out the states then leaves a positive-definite matrix with a (block) tridiagonal inverse [24, Propositions 1 and 2]. They use this structure to describe the closed convex hull of the epigraph [24, Theorems 1 and 2] and, when there are no side constraints, to solve the problem as a shortest-path problem on a directed acyclic graph [24, Proposition 9 and Corollary 3]. Gómez, Han, and Lozano observe that the tridiagonal, tree, and Stieltjes structures behind earlier polynomial cases disappear for bandwidth greater than one [18, Section 1.1]. Section 6 shows that exact optimization is NP-hard already at bandwidth two, even for matrices arbitrarily close to the identity, and that its hard instances cannot be made Stieltjes by changing the signs of variables.

3.2 Approximation, decision diagrams, and convex hulls

Approximation schemes also exploit bounded treewidth. The linear programs of Bienstock and Muñoz [8] (Section 2.3), of polynomial size for each fixed tolerance, replace each continuous variable by a truncated binary expansion. Bienstock and Chen [7, Theorem 1.4] treat convex quadratic problems in which binary indicators control disjoint blocks of variables, when the graph whose vertices are the constraints, with two constraints adjacent if they involve a common block, has bounded treewidth. Their algorithm returns a solution that costs at most the optimal value, satisfies the constraints on the indicators alone exactly, and violates the other constraints by a bounded amount. Like the linear programs of Bienstock and Muñoz, the grid oracle of Lemma 5.1 rounds every coordinate to a grid; it then solves the rounded problem by dynamic programming on the tree decomposition. Theorem 4.3 accepts any restricted additive oracle in its place.

Decision diagrams take a different route. A decision diagram is a layered directed acyclic graph whose layers correspond to the indicator variables. Each path from the root to a terminal node encodes one support; in an exact diagram, the length of the path is the cost of that support, so a shortest path solves the problem [18, Section 2.1]. The nodes are the states of a dynamic program that fixes the indicators one at a time and stores what the rest of the problem needs to know about the partial support. A message, in contrast, conditions on continuous boundary values and minimizes over the internal supports. Gómez, Han, and Lozano [18] build decision diagrams for banded matrices, derive from them extended formulations of the closed convex hull and, from truncated diagrams, an additive approximation scheme, and use the diagrams to re-solve online instances quickly. Choi et al. [11] develop decision diagrams that both solve indicator quadratic problems and give ideal conic extended formulations. Their exact diagrams have polynomial size for fixed rank and for trees and inverse trees with a fixed number of leaves, with $O(n^{k+1})$ nodes for k leaves [11, Section 7]. For each fixed accuracy, their approximate diagrams have size linear in n under polynomial volume growth and a polynomial bound on how many already decided variables

lie within a given graph distance of the undecided ones [11, Definition 15 and Theorem 4]. Section 8 explains how such approximate diagrams can serve as restricted additive oracles.

The diagrams of [18] and [11] are built from the matrix alone, and only their arc lengths depend on the linear coefficients and penalties [18, Remark 1], [11, Definition 7 and Proposition 6]. A representation of this kind can be built once and reused for every linear cost and penalty vector. A message can be reused when data outside its subproblem change (Section 2.2), but not for every cost vector, because it depends on the linear coefficients and perturbed penalties of its internal variables.

Representations built from Q alone can be exponentially large even where optimization is easy. Choi et al. observe this for their own diagrams: on stars, trees in which one center vertex is adjacent to all $n - 1$ other vertices, so that the number of leaves grows linearly with n , their representation becomes exponential in size [11, Section 7.2], and Remark A.3 checks this directly for their state definition and every variable order. Appendix A shows the same for the inverse-principal polytope P_Q , built from the inverses of the principal submatrices of Q , through which Wei et al. [30, Section 3.1 and Theorem 1] describe the closed convex hull of the epigraph of $x^T Q x$ under the indicator constraints. For a sparse, well-conditioned star, every exact linear, second-order-cone, or fixed-order semidefinite lift of P_Q has exponential size (Corollary A.2). This bound concerns P_Q together with its inverse entries; it gives no lower bound for formulations of the epigraph hull that project them out.

3.3 Smoothed analysis and enumeration

Theorem 5.2 is a result of smoothed analysis in the sense of Section 1, with arbitrary data and random perturbations of the penalties only. For binary problems with perturbed linear objectives, two principles used here are already established: exact optimization in expected polynomial time under perturbation, and the conversion of an approximation scheme into such exact optimization. A binary optimization problem has *polynomial smoothed complexity* if some algorithm solves it with running time T satisfying $\mathbb{E}[T^\alpha] \leq \beta \phi \ell$ for constants $\alpha, \beta > 0$, on every instance of input length ℓ whose perturbed coefficients have densities at most $\phi \geq 1$ [3, conference version, Section 1.3]; this is weaker than expected polynomial running time. Beier and Vöcking [3, conference version, Theorem 3] showed that such a problem has polynomial smoothed complexity if and only if it has a randomized pseudopolynomial algorithm, and Röglin and Teng [29, Section 6.2] strengthened this to expected polynomial running time. Dughmi and Roughgarden [14, Proposition 2.4] explicitly convert an approximation scheme into exact smoothed optimization for binary linear objectives.

Lemma 4.1 combines the sandwich inequality, which bounds the size of a family of supports by the number of coordinate sets on which it shows all binary patterns [20, Theorem 5], with the conditioning argument of Beier and Vöcking’s generalized isolating lemma [3, conference version, Lemma 5] (Section 4). Röglin and Teng [29, Lemma 6.1] bound, for a linear objective whose coefficients have bounded densities, the probability that the u th best solution is within ε of the optimum, that is, the probability that at least u solutions lie within ε of the optimum. For linear costs this is a tail bound for the count in Lemma 4.1, proved by a related conditioning argument. Lemma 4.1 bounds the expectation instead, allows an arbitrary deterministic cost for each support, and allows perturbation values of positive probability (atoms), which the finite perturbation grid requires.

Smoothed analysis has also been applied to outputs that, like our dictionaries, describe many objectives at once. Beier, Röglin, Rösner, and Vöcking [4, Theorem 1] bound expected bicriteria Pareto counts with one arbitrary objective and one linear objective whose independent coefficients have bounded densities and finite first moments. Moments of smoothed Pareto counts are also known [10, 29], and smoothed parametric enumeration has been done for shortest paths [1, Section 6]. These

results use different perturbation models and outputs. Theorem 4.3 allows arbitrary deterministic Lipschitz branches and adds $\xi^T z$ to the branch of each support z , with one random ξ shared by all supports and parameter values. The ingredients above are classical. To the best of our knowledge, what is new is the two-sided enumeration guarantee for a restricted additive oracle, which adapts Lawler’s partition method [22] (Lemma 4.2), its use with a parameter net to obtain complete dictionaries on a whole parameter domain (Theorem 4.3), and the specialization of this construction to indicator quadratics (Theorem 5.2).

3.4 Lower bounds

Chains that encode SUBSET SUM through a continuous linear recursion already occur in conditional linear Gaussian inference [25, Section 3]. Theorem 6.1 adapts this mechanism to one positive-definite quadratic with uncoupled indicators and positive penalties, at bandwidth two and near-identity conditioning, in three numerical regimes. Corollary 6.2 carries the hardness over to the perturbed problem by a scaling argument of Beier and Vöcking [3, conference version, Theorem 3].

Hardness of sparse approximation is also known in general. Çivril [12, Theorem 1.1] proves inapproximability for a cardinality-constrained squared-projection objective without graph, positive-definiteness, or spectral restrictions. In the other direction, Das and Kempe [13, Theorem 4.1] give a fully polynomial-time approximation scheme for cardinality-constrained subset selection when the covariance graph of the predictors alone, which corresponds to our matrix graph, has bounded bandwidth. Their guarantee is a relative approximation of explained variance, so it does not conflict with exact hardness of the penalized problem. For fixed bandwidth and eigenvalue bounds, the additive approximation scheme of Gómez, Han, and Lozano runs in time polynomial in the dimension, a bound on the linear coefficients, and the inverse accuracy $1/\varepsilon$ [18, Theorem 1]. Case (1) of Theorem 6.1, with a fixed matrix family, exponentially large coefficients, and a constant gap between the optimal values of YES and NO instances, shows that its dependence on the coefficient bound cannot be replaced by dependence on encoding length unless $P = NP$. Case (2), with $|c_i| \leq 2$ and a possibly exponentially small gap, shows the same for its dependence on $1/\varepsilon$.

Value functions with exponentially many pieces are known in other settings. For parametric linear programs, the optimal value can have exponentially many linear pieces [28, Corollary 1], and exponential growth of piecewise quadratic messages is known for broader nonconvex models [21, Section 5]. Theorem 7.2 realizes such growth in the indicator model with fixed local data and near-identity conditioning. Power-law rates like that of Theorem 7.3 are known for approximation by quadratic bases with a common Hessian [17, Section III, Theorem 3.3], and pruning under uniform error has a covering interpretation [17, Section VI]. Theorem 7.3 gives matching upper and lower exponents for branches whose Hessians differ between supports.

4 Counting and finding near-optimal supports

This section proves the first three ingredients outlined in Section 2.5: the count of near-optimal supports (Lemma 4.1), their enumeration with an approximate oracle (Lemma 4.2), and the passage from single parameter values to a whole parameter domain (Theorem 4.3). Section 4.4 bounds the size of the net for a box and the work per output. These results involve no quadratic functions and no graph, so we state them for an arbitrary family of supports.

Let $Z \subseteq \{0, 1\}^m$ be nonempty. We call its elements *supports*, identifying z with the set $\{i : z_i = 1\}$. Each support has a deterministic cost plus a random linear perturbation $\xi^T z$. For a message $M_{I,S}$, the set Z is $\{0, 1\}^I$, the deterministic cost of a support A at a boundary value t is its unperturbed

branch $q_A(t)$ from (8), and adding the perturbation gives the perturbed branch (9). If $m = 0$, then Z contains only the empty support, and all statements hold with empty products equal to one.

4.1 Expected number of near-optimal supports

Fix the parameter for now, so that each support has a single cost. Assume that the coordinates ξ_i of the perturbation are independent real random variables satisfying

$$\mathbb{P}\{\xi_i \in J\} \leq \phi|J| + \tau \quad \text{for every closed interval } J, \quad \phi > 0, \quad \tau \geq 0. \quad (12)$$

The constant ϕ acts as a density bound: a density bounded by ϕ gives (12) with $\tau = 0$. The term τ allows atoms. Atoms must be allowed because the perturbation grid used later is finite, so exact ties occur with positive probability. For deterministic costs $g_z \in \mathbb{R}$, which may be arbitrary but must not depend on ξ , put

$$F_z = g_z + \xi^T z, \quad v = \min_{z \in Z} F_z, \quad \mathcal{N}_a = \{z \in Z : F_z \leq v + a\}, \quad a \geq 0.$$

Thus $\mathcal{N}_a$ is the set of supports within a of the optimum, and $\mathcal{N}_0$ is the set of optimal supports.

Without the perturbation, $\mathcal{N}_a$ can be all of Z ; for instance, every support is optimal when all costs g_z are equal. The perturbation breaks such ties, and in this extreme case one can see exactly how. Let $Z = \{0, 1\}^m$ and $g_z = 0$ for all z . Then

$$F_z - v = \sum_{i=1}^m (\xi_i z_i - \min\{\xi_i, 0\}).$$

The i th term equals $|\xi_i|$ if $z_i = 1$ and $\xi_i > 0$, or if $z_i = 0$ and $\xi_i < 0$; otherwise it is zero. A support in $\mathcal{N}_a$ must therefore take the cheaper value of z_i on every coordinate with $|\xi_i| > a$, so $|\mathcal{N}_a| \leq 2^s$, where s is the number of coordinates with $|\xi_i| \leq a$. By independence and (12),

$$\mathbb{E} 2^s = \prod_{i=1}^m (1 + \mathbb{P}\{|\xi_i| \leq a\}) \leq (1 + 2\phi a + \tau)^m.$$

Many near-optimal supports would need many coordinates with a small perturbation, and that is unlikely when a is small. Lemma 4.1 below proves the same bound for an arbitrary family Z and arbitrary costs g_z .

For general costs, the near-optimal supports have no such coordinatewise description, but the same mechanism works one coordinate at a time. Fix a coordinate i and all coordinates of ξ other than ξ_i . Among the supports with $z_i = 0$, the least perturbed cost is then a fixed number; among those with $z_i = 1$, it is a fixed number plus ξ_i . If both groups contain a support in $\mathcal{N}_a$, both least costs lie in $[v, v + a]$, so they differ by at most a . This confines ξ_i to a fixed interval of length $2a$, which has probability at most $2\phi a + \tau$. Splitting by one bit and conditioning on the other coordinates of the perturbation is the conditioning argument of Beier and Vöcking's generalized isolating lemma [3, conference version, Lemma 5] (compare [29, Lemma 6.1]); we apply it to several coordinates at once. To turn such events into a bound on the size of $\mathcal{N}_a$, we need to know that a large family of supports must vary freely on many sets of coordinates. Shattering makes this precise.

A family $\mathcal{F} \subseteq \{0, 1\}^m$ *shatters* a coordinate set $D \subseteq [m]$ if the restrictions z_D , $z \in \mathcal{F}$, include all $2^{|D|}$ binary patterns. Write $\text{Sh}(\mathcal{F})$ for the collection of coordinate sets shattered by $\mathcal{F}$. Every nonempty family shatters the empty set, and $\mathcal{F}$ shatters $\{i\}$ exactly when it contains supports with $z_i = 0$ and with $z_i = 1$. For example, the family $\{00, 10, 01\}$ in $\{0, 1\}^2$ shatters $\emptyset$, $\{1\}$, and $\{2\}$, but

not $\{1, 2\}$, because the pattern 11 is missing. The sandwich inequality, in the form stated by Kozma and Moran [20, Theorem 5], says that a family shatters at least as many sets as it has members:

$$|\mathcal{F}| \leq |\text{Sh}(\mathcal{F})|. \quad (13)$$

In the example both sides equal three. The family $\{00, 11\}$ shatters the same three sets, so the inequality can be strict.

For completeness, we recall the induction on m . Split $\mathcal{F}$ by its last coordinate into projected families $\mathcal{F}^0, \mathcal{F}^1 \subseteq \{0, 1\}^{m-1}$, and set $\mathcal{F}^\cup = \mathcal{F}^0 \cup \mathcal{F}^1$ and $\mathcal{F}^\cap = \mathcal{F}^0 \cap \mathcal{F}^1$, so that $|\mathcal{F}| = |\mathcal{F}^\cup| + |\mathcal{F}^\cap|$. Every set shattered by $\mathcal{F}^\cup$ is shattered by $\mathcal{F}$. If $\mathcal{F}^\cap$ shatters D , then $\mathcal{F}$ shatters $D \cup \{m\}$. The two collections are disjoint, because only the second contains m . With the induction hypothesis for $\mathcal{F}^\cup$ and $\mathcal{F}^\cap$, this gives $|\mathcal{F}| \leq |\text{Sh}(\mathcal{F}^\cup)| + |\text{Sh}(\mathcal{F}^\cap)| \leq |\text{Sh}(\mathcal{F})|$. In the base case $m = 0$, the family $\mathcal{F}$ is either empty, and shatters no set, or a single point, and shatters only the empty set; in both cases $|\mathcal{F}| = |\text{Sh}(\mathcal{F})|$.

By (13), a large set $\mathcal{N}_a$ must shatter many coordinate sets, so it suffices to bound the probability that $\mathcal{N}_a$ shatters a fixed set D .

Lemma 4.1 (Near-optimal count). *Under (12), for every $a \geq 0$,*

$$\mathbb{E}|\mathcal{N}_a| \leq (1 + 2\phi a + \tau)^m. \quad (14)$$

Here $|\mathcal{N}_a|$ counts every support in $\mathcal{N}_a$, including tied ones.

Proof. Fix $D \subseteq [m]$ and condition on $(\xi_i)_{i \notin D}$. For a pattern $u \in \{0, 1\}^D$, define

$$C_u = \min_{z \in Z: z_D = u} \left(g_z + \sum_{i \notin D} \xi_i z_i \right).$$

If some class $\{z \in Z : z_D = u\}$ is empty, then Z , and hence $\mathcal{N}_a$, does not shatter D . Otherwise every C_u is finite and independent of ξ_D , and the smallest perturbed cost in class u is $C_u + \sum_{i \in D} \xi_i u_i$. If $\mathcal{N}_a$ shatters D , each class contains a support in $\mathcal{N}_a$, so this smallest cost lies in $[v, v + a]$ for every u . Let $\chi_i \in \{0, 1\}^D$ have a single one, in coordinate i . Comparing $u = 0$ with $u = \chi_i$ gives

$$|C_{\chi_i} + \xi_i - C_0| \leq a \quad (i \in D).$$

Given $(\xi_i)_{i \notin D}$, each condition places ξ_i in a fixed interval of length $2a$, and the coordinates ξ_i , $i \in D$, are independent. By (12), the conditional probability that $\mathcal{N}_a$ shatters D is at most $(2\phi a + \tau)^{|D|}$, and hence so is $\mathbb{P}\{D \in \text{Sh}(\mathcal{N}_a)\}$. Taking expectations in (13) with $\mathcal{F} = \mathcal{N}_a$ and summing over all $D \subseteq [m]$ gives $\mathbb{E}|\mathcal{N}_a| \leq \sum_D (2\phi a + \tau)^{|D|} = (1 + 2\phi a + \tau)^m$. $\square$

Section 5 draws each ξ_i uniformly from the perturbation grid (11), whose N points are equally spaced in $[-\sigma, \sigma]$ with spacing $2\sigma/(N - 1)$; the bounds below hold for any $N \geq 2$. An interval of length d contains at most $d(N - 1)/(2\sigma) + 1$ grid points, so its probability is at most $d/(2\sigma) + 1/N$. Hence

$$\phi = \frac{1}{2\sigma}, \quad \tau = \frac{1}{N} \quad (15)$$

satisfy (12). Each grid point is an atom of probability $1/N$, so the grid needs many points: Theorem 4.3 below requires $\tau \leq 1/(2n)$, which is why Theorem 5.2 takes $N \geq 2n$.

4.2 Certified enumeration with a restricted additive oracle

Still at a fixed parameter value, we now want to list every support in $\mathcal{N}_\delta$, not only bound their expected number. The search below divides the supports into groups by fixing some of their coordinates, so the oracle must be able to optimize with coordinates of z fixed. It may be approximate, because the oracle of Section 5 solves a grid version of the problem.

A partial assignment p fixes some coordinates of z . Its *cell* Z_p is the set of supports in Z that agree with p . A *restricted additive oracle* with accuracy $\varepsilon > 0$ accepts any partial assignment p . If $Z_p = \emptyset$, it reports this; otherwise it returns a support $z_p \in Z_p$ together with its exact cost $V_p = F_{z_p}$, where

$$\min_{z \in Z_p} F_z \leq V_p \leq \min_{z \in Z_p} F_z + \varepsilon. \quad (16)$$

The returned support may be suboptimal by up to ε , but its reported cost is exact. Hence $\ell_p = V_p - \varepsilon$ is a certified lower bound on all costs in Z_p .

The search keeps the supports not yet output partitioned into cells. When it processes a cell, it outputs the support z^{out} returned by the oracle and splits the rest of the cell according to the first free coordinate at which a support differs from z^{out} . For example, let $m = 3$, let p fix nothing, and let $z^{\text{out}} = 101$. Writing $*$ for a free coordinate, the other seven supports split into the cells $0**$, $11*$, and 100 : a support that differs from 101 in the first coordinate lies in the first cell, one that agrees there but differs in the second lies in the second cell, and the only remaining support is 100. Each piece is again a cell, so the oracle can be called on it.

Given a tolerance $\delta \geq 0$, *certified enumeration* proceeds as follows; its stopping test uses only the certified lower bounds ℓ_p .

1. Query the oracle on the empty assignment, whose cell is Z , and record its value U , which stays fixed.
2. Keep a heap of nonempty cells keyed by ℓ_p , each stored with its returned support z_p . Start with the cell Z .
3. While the heap is nonempty and its smallest key is at most $U + \delta$, extract a cell Z_p with the smallest key and output $z^{\text{out}} = z_p$. Let $j_1 < \dots < j_r$ be the coordinates left free by p . For $s = 1, \dots, r$, form the child cell that keeps the fixed coordinates of p and sets

$$z_{j_h} = z_{j_h}^{\text{out}} \quad (h < s), \quad z_{j_s} = 1 - z_{j_s}^{\text{out}}. \quad (17)$$

Query the oracle on each child and insert the nonempty ones.

4. Return the set E of output supports.

The children in (17) are the partition step of Lawler's ranked enumeration [22]. Lawler uses an exact oracle and lists solutions in order of cost. With an additive oracle the outputs are not ranked; the certified stopping test gives the two-sided inclusion (18) instead.

Lemma 4.2 (Enumeration guarantee). *Certified enumeration terminates, outputs no support twice, and returns a set E satisfying*

$$\mathcal{N}_\delta \subseteq E \subseteq \mathcal{N}_{\delta+2\varepsilon}. \quad (18)$$

It uses at most $1 + m|E|$ oracle calls.

Proof. Each support in Z_p other than z^{out} has a first free coordinate j_s at which it differs from z^{out} , so it lies in exactly one child (17). Hence the cells in the heap always partition the supports not yet output; empty cells contain nothing and are dropped. Outputs are therefore distinct, and the procedure terminates because Z is finite.

The value U is the cost of a support, so $v \leq U$. Suppose a support z with $F_z \leq v + \delta$ has not been output. It lies in some heap cell Z_p , and $\ell_p \leq \min_{y \in Z_p} F_y \leq F_z \leq v + \delta \leq U + \delta$. Thus the procedure cannot stop while z remains, which proves $\mathcal{N}_\delta \subseteq E$. Conversely, (16) for the first query gives $U \leq v + \varepsilon$. An output comes from a cell with $\ell_p \leq U + \delta$, so

$$F_{z_p} = V_p = \ell_p + \varepsilon \leq U + \delta + \varepsilon \leq v + \delta + 2\varepsilon.$$

Finally, there is one call for the first query and at most $r \leq m$ calls for the children of each output. $\square$

The inclusions (18) hold for every realization of ξ and for every oracle answer permitted by (16), including the worst one. This is what allows Lemma 4.1 to bound the output: its probability bounds concern the sets $\mathcal{N}_a$, which are defined without reference to the search, so they never condition on the adaptive search history.

4.3 A complete dictionary on a parameter domain

Now let the costs depend on a parameter. Let T be a nonempty compact metric space, and let deterministic functions $q_z : T \rightarrow \mathbb{R}$ be L -Lipschitz for a common $L \geq 0$. Define the *branches* and their minimum

$$F_z(t) = q_z(t) + \xi^T z, \quad V(t) = \min_{z \in Z} F_z(t).$$

A *dictionary* for V on T is a set of branches whose pointwise minimum equals V on T . In Section 5, T is a separator box and V is a message. For $t \in T$ and $a \geq 0$, let $\mathcal{N}_a(t)$ denote the set $\mathcal{N}_a$ defined with the costs $g_z = q_z(t)$. A *parameter net* with radius r is a finite set $t_1, \dots, t_J \in T$ such that every point of T lies within distance r of some t_j .

A support that is optimal somewhere on T may be optimal only on a region that contains no net point, or only at a single tie point, so the theorem below enumerates near-optimal supports at each net point. The net must be fixed before ξ is drawn. At a point chosen after seeing ξ , the costs depend on ξ and Lemma 4.1 does not apply. For instance, a point where two branches tie for the minimum has at least two optimal supports by construction.

Expectations of the sizes of random sets of supports require these sets to depend measurably on ξ . The set of supports optimal somewhere on T does so automatically: a support z is optimal somewhere exactly when $\min_{t \in T} (F_z(t) - V(t)) = 0$, and this minimum is continuous in ξ because T is compact. For the enumeration outputs E_j below, we assume that the oracle's returned support is a measurable function of ξ and that all remaining ties, such as equal heap keys, are broken by fixed deterministic orders.

Theorem 4.3 (Complete dictionary from restricted approximation). *Suppose (12) holds, $n \geq \max\{1, m\}$ is an integer, and $\tau \leq 1/(2n)$. Set*

$$\varepsilon = \delta = \frac{1}{12n\phi}. \quad (19)$$

For $L > 0$, take a deterministic parameter net $t_1, \dots, t_J$ in T with radius $\varepsilon/(2L)$. For $L = 0$, take $J = 1$ and any point $t_1 \in T$. At every t_j , run certified enumeration with tolerance δ for the costs

$F_z(t_j)$, using a restricted additive oracle with accuracy ε . Let E_j be its output and $\mathcal{D} = \bigcup_j E_j$. Then, for every realization of ξ ,

$$\bigcup_{t \in T} \arg \min_{z \in Z} F_z(t) \subseteq \mathcal{D}, \quad V(t) = \min_{z \in \mathcal{D}} F_z(t) \quad (t \in T). \quad (20)$$

Moreover,

$$\mathbb{E} \sum_{j=1}^J |E_j| \leq eJ, \quad \mathbb{E}[\text{number of oracle calls}] \leq J(1 + em). \quad (21)$$

The branches F_z , $z \in \mathcal{D}$, thus form a dictionary for V on T . By the inclusion and the equality in (20), at every $t \in T$, including tie points and points on the boundary of T , the minimizers over $\mathcal{D}$ are exactly the minimizers over Z . The integer n only sets the tolerance; Section 5 takes it to be the problem dimension, so that one accuracy serves every message.

Proof. Fix a realization of ξ , and let z be a support that minimizes $F_z(t)$ over Z at some $t \in T$. The term $\xi^T z$ does not depend on t , so every branch is L -Lipschitz. First suppose $L > 0$. Choose t_j within distance $\varepsilon/(2L)$ of t , and let y minimize at t_j . Then

$$F_z(t_j) \leq F_z(t) + \varepsilon/2 \leq F_y(t) + \varepsilon/2 \leq F_y(t_j) + \varepsilon.$$

Thus $z \in \mathcal{N}_\varepsilon(t_j) = \mathcal{N}_\delta(t_j)$, and Lemma 4.2 gives $z \in E_j$. If $L = 0$, every branch is constant, so z is optimal at t_1 and lies in E_1 . This proves the inclusion in (20). Every element of $\mathcal{D}$ lies in Z , so $\min_{z \in \mathcal{D}} F_z(t) \geq V(t)$, and the inclusion gives equality.

For the expectations, fix j . The point t_j is deterministic, so the costs $q_z(t_j)$ are deterministic and Lemma 4.1 applies. By Lemma 4.2, the outputs at t_j lie within $\delta + 2\varepsilon = 3\varepsilon$ of the optimum, and (19) makes $6\phi\varepsilon = 1/(2n)$. With $\tau \leq 1/(2n)$ and $m \leq n$, Lemma 4.1 gives

$$\mathbb{E}|E_j| \leq \mathbb{E}|\mathcal{N}_{3\varepsilon}(t_j)| \leq (1 + 6\phi\varepsilon + \tau)^m \leq (1 + 1/n)^m \leq e.$$

Summing over j gives the first bound in (21). Lemma 4.2 gives at most $1 + m|E_j|$ oracle calls at t_j , so the expected number of calls is at most $J(1 + em)$. The same ξ is used at every net point; linearity of expectation needs no independence between points. $\square$

4.4 Net size and work per output

Let $T = [-R, R]^k$ with the Euclidean metric and $k, R, L > 0$. Divide each coordinate interval into $h = \max\{1, \lceil 2kRL/\varepsilon \rceil\}$ equal intervals and use the midpoints of the resulting h^k subboxes. Every point of T lies within distance $\sqrt{k}R/h \leq kR/h \leq \varepsilon/(2L)$ of a midpoint, and

$$J = h^k \leq (1 + 2kRL/\varepsilon)^k. \quad (22)$$

If $k = 0$ or $R = 0$, the box is a single point. Rational R, L, ε give rational net points of polynomial encoding length. At fixed k , the bound (22) is a polynomial in R, L , and $1/\varepsilon$.

Theorem 4.3 bounds the expected numbers of outputs and oracle calls. The remaining work is bookkeeping, and it costs at most a polynomial factor per output or oracle call, even for a realization with exponentially many outputs. At one net point, each of the at most 2^m outputs creates at most m cells, so at most $1 + m2^m$ cells are created and each heap operation uses $O(m + \log(m + 1))$ comparisons. Supports from different net points can be deduplicated in a binary trie in $O(m)$ time per output. Because these factors do not grow with the number of outputs, only the first moment of the output count is needed. An algorithm that outputs branches must also construct each branch from its support. Hence, if exact evaluation of $F_z(t_j)$, construction of a branch from its support, and each restricted oracle call have uniformly polynomial bit cost, the expected bit cost is polynomial whenever J is polynomial.

5 Exact messages at fixed treewidth

Section 2.5 outlined the proof of Theorem 5.2. This section completes it by meeting the two requirements of Theorem 4.3 for indicator quadratics: a Lipschitz constant common to all branches and a restricted additive oracle. Section 5.1 bounds all conditional optimizers uniformly, which gives the Lipschitz constant, and Section 5.2 uses these bounds to build the oracle. Section 5.3 proves Theorem 5.2, and Section 5.4 bounds the original optimum.

5.1 Uniform bounds on conditional optimizers

Both requirements need bounds that hold for all supports at once. The net of Theorem 4.3 has radius $\varepsilon/(2L)$, so L must be a Lipschitz constant for every branch. The oracle will round a restricted optimizer to a finite grid, and the rounded point is close to the optimizer only if the grid covers a range that is known in advance to contain it. The oracle does not know which support is optimal, so this range must contain the conditional optimizers of all supports.

Let $k = |S|$ and $A \subseteq I$. Principal submatrices inherit (2), so $\|Q_{AA}^{-1}\|_2 \leq 1/\mu$. Moreover $\|Q_{AS}\|_2 \leq \|Q\|_2 \leq H$, $\|c_A\|_2 \leq C\sqrt{n}$, and $\|t\|_2 \leq \sqrt{k}R$ on $[-R, R]^k$. Inserting these bounds into (7), and into the gradient of (8), shows that every $t \in [-R, R]^k$ satisfies

$$\|x_A(t)\|_2 \leq \frac{C\sqrt{n} + 2H\sqrt{k}R}{2\mu} \leq X_k := \frac{nC + 2HkR}{2\mu}, \quad (23)$$

$$\|\nabla q_A(t)\|_2 = \|2Q_{SA}x_A(t)\|_2 \leq L_k := 2HX_k. \quad (24)$$

The second bound in (23) replaces the square roots by a rational quantity, which can be computed exactly. Since the box is convex, (24) makes every q_A , and hence every F_A , L_k -Lipschitz on it. The bounds are uniform over supports, so they still hold when some indicator bits are fixed: a restricted optimum is the conditional optimizer of some support.

The bounds grow with R because the boundary values enter the linear term $c_A + 2Q_{AS}t$ of the conditional problem. For the same reason, a bound on the global optimizer alone would not suffice: it need not bound the conditional optimizers for a prescribed boundary value.

The same estimate also shows how large the box must be to contain every optimal boundary value. For every ξ , each optimal solution of (1) is the conditional optimizer of its support for $I = [n]$ and $S = \emptyset$, so (23) with $k = 0$ gives $\|x\|_2 \leq C\sqrt{n}/(2\mu) \leq nC/(2\mu)$. Hence, for every separator S , the box with rational radius $R = nC/(2\mu)$ contains the boundary values x_S of all optimal solutions.

5.2 A restricted grid oracle

Fix a message $M_{I,S}$, a rational boundary point $t \in [-R, R]^k$, and an accuracy $\varepsilon > 0$. The oracle must approximately minimize the internal objective of (6) at t under arbitrary fixed indicator bits, as (16) requires. If the continuous variables could take only finitely many values, this would be a discrete problem whose costs each involve one vertex or one edge of the support graph, and dynamic programming over the tree decomposition would solve it exactly. We therefore restrict every coordinate to a finite grid, fine enough that rounding costs at most ε , solve the grid problem exactly, and return the support of the grid solution with its exact continuous cost.

First suppose $X_k > 0$. Let K be the least positive even integer satisfying

$$K^2\varepsilon \geq HnX_k^2. \quad (25)$$

Then $K \leq 2 + X_k\sqrt{Hn/\varepsilon}$. Use the coordinate grid

$$G_K = \{-X_k + 2X_kj/K : 0 \leq j \leq K\}.$$

Its $K + 1$ points are spaced $2X_k/K$ apart, so every point of $[-X_k, X_k]$ lies within X_k/K of the grid. The proof of Lemma 5.1 shows that (25) then makes the cost of rounding at most ε . Each vertex $i \in I$ has an inactive state $(z_i, x_i) = (0, 0)$ and active states $(1, a)$ for $a \in G_K$. Since K is even, $0 \in G_K$: as in the continuous problem, an active coordinate may be zero, and this state is distinct from the inactive state. Fixed bits delete incompatible states. For example, $K = 4$ gives $G_4 = \{-X_k, -X_k/2, 0, X_k/2, X_k\}$ and six states per vertex; fixing $z_i = 1$ leaves the five active states, and fixing $z_i = 0$ leaves only the inactive one.

The grid objective is the sum of the unary costs

$$Q_{ii}x_i^2 + (c_i + 2(Q_{Ist})_i)x_i + (\lambda_i + \xi_i)z_i$$

and the edge costs $2Q_{ij}x_ix_j$ for $i < j$ in I with $Q_{ij} \neq 0$, evaluated at grid states. These are the terms of the internal objective of (6) at t ; the boundary enters only through the linear coefficients.

The dynamic program works on the rooted decomposition of Section 2.3. Restrict every bag to I , and write $B_u^I = B_u \cap I$ for the restricted bag of node u . Restricting the bags to I preserves the defining conditions of a tree decomposition, so the restricted bags form a tree decomposition of the subgraph induced by I ; empty bags are harmless. Assign each unary and edge cost to exactly one bag containing its variables. For a bag u and an assignment s of states to its vertices, let $a_u(s)$ be the sum of its assigned costs, and let $D_u(s)$ be the minimum total of the costs assigned to the subtree of u , over assignments that agree with s on B_u^I . The recurrence is

$$D_u(s) = a_u(s) + \sum_{v \text{ child of } u} \min_{r: r|_{B_u^I \cap B_v^I = s|_{B_u^I \cap B_v^I}} D_v(r). \quad (26)$$

The inner minimum is a discrete analogue of a message: for each assignment of states to the shared vertices $B_u^I \cap B_v^I$, it gives the least total of the costs assigned to the subtree of v . Because each vertex has finitely many states, it is a finite table rather than a minimum of quadratics in continuous variables. The running-intersection property makes the internal sets of different children disjoint, and it ensures that agreement on $B_u^I \cap B_v^I$ is sufficient for compatibility. Induction from the leaves shows that (26) computes D_u , and minimizing D at the root gives the exact grid optimum.

Each child table is first minimized over its states at each assignment to the shared vertices $B_u^I \cap B_v^I$; reading these projected tables at the parent avoids a product of two bag tables. A restricted bag has at most $w + 1$ vertices with at most $K + 2$ states each, so each table has at most $(K + 2)^{w+1}$ entries. Each entry adds its assigned costs and one projected value per child, and each child table is projected once. With b bags and $O(n^2)$ costs, the number of arithmetic operations is

$$O((b + n^2)(K + 2)^{w+1}). \quad (27)$$

Back-pointers recover a minimizing grid solution and its support. The bound needs no balance or degree condition on the decomposition.

Lemma 5.1 (Restricted additive oracle). *Under any fixed-bit restriction, solve the grid problem and let A be the support of its optimal solution. Returning A with its exact perturbed value $F_A(t)$ from (9) satisfies (16).*

Proof idea. Rounding a true restricted optimizer to the grid raises its cost by at most ε , because the linear part of the rounding error vanishes at a minimizer. The grid optimum is at least as good, and reoptimizing its support continuously can only lower its cost.

Proof. Let A^* be an optimal support of the restricted continuous problem and x^* its unique continuous minimizer, which is $x_{A^*}(t)$ extended by zeros. By (23), $x^* \in [-X_k, X_k]^I$. Round each

active coordinate to a nearest point of G_K and keep the inactive coordinates zero. The rounding error e is supported on A^* and has $|e_i| \leq X_k/K$, so $\|e\|_2^2 \leq nX_k^2/K^2$. Since x^* is stationary on A^* , the linear term in e cancels, and the objective increases by

$$e^T Q_{II} e \leq H\|e\|_2^2 \leq HnX_k^2/K^2 \leq \varepsilon$$

by (25). The rounded point has support A^* , so it obeys the fixed bits, and the optimal grid solution costs no more. Its support A also obeys the fixed bits, and continuous reoptimization on A can only decrease the cost, so $F_A(t)$ is at most the grid optimum. Hence $F_A(t)$ lies between the restricted optimum and the restricted optimum plus ε . $\square$

If $X_k = 0$, then (23) forces every conditional optimizer to be zero, so $F_A(t) = \sum_{i \in A} (\lambda_i + \xi_i)$. An exact oracle then keeps the fixed bits and sets each free bit to minimize its own perturbed penalty $\lambda_i + \xi_i$.

The construction and the lemma use X_k only to ensure that $x^* \in [-X_k, X_k]^I$. They therefore remain valid when X_k is replaced throughout by any larger rational bound, which lets Theorem 5.2 use one common bound for all messages. Every fixed-bit restriction is feasible, because an active coordinate may be zero, so the oracle never reports an empty cell.

5.3 The algorithm and its expected bit complexity

Let the supplied decomposition have width w and b bags, and let the box radius $R \geq 0$ and the perturbation scale $\sigma > 0$ be rational. The algorithm uses one perturbation draw and one set of parameters for all messages, chosen so that Theorem 4.3 and Lemma 5.1 apply to every message at once. Every separator has $|S| \leq w + 1$, and X_k and L_k do not decrease with k , so it suffices to evaluate (23) and (24) at $k = w + 1$. With $\phi = 1/(2\sigma)$ from (15), the accuracy (19) becomes $\sigma/(6n)$. This gives the parameters

$$X = \frac{nC + 2H(w+1)R}{2\mu}, \quad L = 2HX, \quad \varepsilon = \frac{\sigma}{6n}, \quad J_* = \left(1 + \frac{12n(w+1)RL}{\sigma}\right)^{w+1}, \quad (28)$$

$K = \text{the least positive even integer with } K^2\varepsilon \geq HnX^2.$

Thus K is defined by (25) with X in place of X_k , and $K = 2$ if $X = 0$. By (22) with $\varepsilon = \sigma/(6n)$ and $k \leq w + 1$, J_* bounds the number of points of the midpoint net on every box.

For each message, the algorithm runs certified enumeration (Lemma 4.2) with tolerance ε at every point of the midpoint parameter net of the box (Section 4.4), using the oracle of Lemma 5.1 with X in place of X_k . It keeps the union of the supports found and builds the perturbed branch F_A of each. Every oracle call runs the grid dynamic program on all of I .

Theorem 5.2 (Exact messages at fixed treewidth). *Fix w . Assume (1)–(2), a supplied tree decomposition of width w with b bags, and rational $R \geq 0$ and $\sigma > 0$. Let N be the smallest power of two with $N \geq 2n$, and let $\xi_1, \dots, \xi_n$ be independent and uniform on the perturbation grid (11). There is an algorithm with the following properties.*

1. *For every realization of ξ , it returns, for every message $M_{I,S}$ of Section 2.4, a dictionary of quadratics with rational coefficients on the box $[-R, R]^{|S|}$. Each dictionary contains every support that is optimal somewhere on its box. The algorithm also returns a global optimizer of the perturbed problem (1).*
2. *It uses $n \log_2 N = O(n \log(n+1))$ random bits.*

3. Let ℓ be the total binary input length, and let X , J_* , and K be the parameters (28). The expected number of bit operations of the algorithm is at most

$$P_w(\ell) b J_* (K + 2)^{w+1}, \quad K + 2 \leq 4 + X n \sqrt{6H/\sigma}, \quad (29)$$

where P_w is a polynomial that depends only on w .

In particular, at fixed w the expected bit complexity is polynomial in ℓ and in the numerical quantities $C, H, \mu^{-1}, R, \sigma^{-1}$.

The factors in (29) follow the structure of the algorithm: at most b messages, at most J_* net points per message, an expected $O(n)$ oracle calls per net point (absorbed into $P_w(\ell)$), and $(K + 2)^{w+1}$ states per bag in each call. Each dictionary is the union of the enumeration outputs at the net points of its box, so by (21) its expected size is at most eJ_* .

Proof idea. Correctness for every realization of ξ follows from Theorem 4.3 and Lemma 5.1. The rest of the proof checks that all numbers have polynomial bit length, so that operation counts become bit counts, and sums the expected oracle calls over the messages.

Proof. Step 1: correctness. Fix a message $M_{I,S}$, so $k = |S| \leq w + 1$. Apply Theorem 4.3 with $Z = \{0, 1\}^I$, $m = |I| \leq n$, $T = [-R, R]^k$, the deterministic functions q_A of (8) (so that the perturbed costs are the branches F_A), $\phi = 1/(2\sigma)$, and $\tau = 1/N$. These values satisfy (12) by (15), the choice of N gives $\tau \leq 1/(2n)$, and (19) gives $\varepsilon = \sigma/(6n)$. Since $k \leq w + 1$, the common bounds X and L dominate (23) and (24), so every q_A is L -Lipschitz on the box. Use the midpoint net of (22), or one point if the box is a singleton or $L = 0$. Its cardinality is at most $(1 + 12nkRL/\sigma)^k \leq J_*$. At each net point, the oracle is Lemma 5.1 with X in place of X_k , or the exact oracle for the case $X = 0$. Ties in the dynamic program and in the enumeration are broken by fixed orders. Since ξ takes finitely many values, the measurability required by Theorem 4.3 holds automatically. Let $\mathcal{D}$ be the union of the outputs. By (20) and (10), the branches F_A , $A \in \mathcal{D}$, form a dictionary for $M_{I,S}$ on the box that contains every support optimal somewhere on it. This holds for every realization. For $M_{[n],\emptyset}$ the branches are constants. The smallest one gives a global minimizing support, and (7) gives its rational optimizer.

Step 2: bit lengths of parameters and dynamic-program values. Every input rational has bit length at most ℓ , and $n, b \leq O(\ell)$. The quantities X , L , ε , the subdivision count of the net, the net coordinates, and the sampled penalties are formed by sums, products, reciprocals of positive rationals, and integer ceilings. Hence they have polynomial bit length. Binary search with exact rational comparisons finds K in polynomially many bit operations, and $\log K$ is polynomial; no square root is computed. Every grid point is an integer multiple of X/K with absolute value at most X . Hence each unary or edge cost takes all its grid values with one fixed denominator and a bounded magnitude, both of polynomial bit length. A finite entry of (26) is a sum of values of a subset of these $O(n^2)$ costs, each used at most once. The product of the cost denominators is a common denominator, and both it and the entry's magnitude have polynomial bit length. Minima select values and create no new denominators. Hence all additions and comparisons in the dynamic program have polynomial bit cost, for every realization of ξ , including realizations with large enumerations.

Step 3: branch construction. Each exact value $F_A(t)$ returned by the oracle, and each branch F_A , comes from a principal system $Q_{AA}u = d$. Clearing denominators gives an integer system whose entries have polynomial bit length. Each determinant is a sum of at most $n!$ products of at most n such integers, so its bit length is polynomial in ℓ . Since Q_{AA} is nonsingular, exact rational elimination gives the coefficients of (7) and (8) with polynomial bit length in polynomial bit time.

Adding the rational perturbations gives the same bound for F_A . The affine map (7) can be stored with each branch to recover conditional optimizers.

Step 4: expected work. By (21) with $J \leq J_*$ and $m \leq n$, the expected number of oracle calls is at most $J_*(1 + en)$ per message, hence at most $bJ_*(1 + en)$ in total. Linearity of expectation applies even though the messages share ξ and their subproblems overlap. Each call uses (27) arithmetic operations, each of polynomial bit cost by Step 2. Heap operations, deduplication, exact support evaluations, and branch construction add a polynomial factor per output or call (Section 4.4), and each output comes from a call. The factors $b + n^2$ and $1 + en$ and these polynomial factors are absorbed into $P_w(\ell)$. Finally, (25) gives $K \leq 2 + X\sqrt{Hn/\varepsilon}$, and $\varepsilon = \sigma/(6n)$ turns this into the bound on $K + 2$ in (29). $\square$

The dictionaries are exact on the box and need not be exact outside it, where the least retained branch may exceed the message. The randomness affects only the running time. For exceptional realizations of ξ the dictionaries may be exponentially large, but the algorithm still terminates with exact output.

5.4 An additive certificate for the original objective

Theorem 5.2 is exact for the perturbed penalties $\lambda + \xi$, but the original problem has penalties λ . Let v_0 be its optimum, the value of (1) at $\xi = 0$. The perturbation changes the cost of each solution (x, z) by $\xi^T z$, and $|\xi_i| \leq \sigma$, so an exact perturbed optimizer is also nearly optimal for the original problem. Let $(\hat{x}, \hat{z})$ be any exact optimizer of the perturbed problem, however computed, and define

$$\text{LB} = v_\xi - \sum_i \max\{\xi_i, 0\}, \quad \text{UB} = v_\xi - \xi^T \hat{z}. \quad (30)$$

For every feasible (x, z) , the unperturbed value equals the perturbed value minus $\xi^T z$. The perturbed value is at least v_ξ and $\xi^T z \leq \sum_i \max\{\xi_i, 0\}$, so the unperturbed value is at least LB. At $(\hat{x}, \hat{z})$ the unperturbed value equals UB. Hence

$$\text{LB} \leq v_0 \leq \text{UB}, \quad \text{UB} - \text{LB} = \sum_i (\max\{\xi_i, 0\} - \xi_i \hat{z}_i) \leq \sum_i |\xi_i| \leq n\sigma. \quad (31)$$

The bracket holds for every realization of ξ , and its width is additive, not relative. In particular, $(\hat{x}, \hat{z})$ is a feasible solution of the original problem whose cost UB exceeds v_0 by at most $n\sigma$. This makes the role of σ explicit: a smaller σ narrows the bracket, while the bound (29) grows polynomially with σ^{-1} .

The certificate is not a cheaper way to approximate v_0 . For any $\eta > 0$, one call of the oracle of Lemma 5.1 with $\xi = 0$, $I = [n]$, $S = \emptyset$, and accuracy η deterministically returns a support whose exact cost lies in $[v_0, v_0 + \eta]$. The certificate matters when exact perturbed messages or an exact perturbed solution are computed anyway; it then brackets the original optimum at no extra cost.

6 Exact hardness at width two

The bound of Theorem 5.2 is polynomial in the values of the numerical bounds $C, H, \mu^{-1}, R, \sigma^{-1}$, not in their bit lengths, so it is exponential in the input length when, for example, C is exponentially large. This section shows that some numerical dependence of this kind is necessary. Theorem 6.1 proves that exact optimization is NP-complete already for matrices of bandwidth two that are arbitrarily close to the identity. Bandwidth one is different: the support graph is then a union

of paths, and the path algorithm of [26] and the tree algorithm of [6] solve the problem with quadratically many arithmetic operations.

Section 6.3 carries the hardness over to the perturbed problem (Corollary 6.2), and Section 3 compares both results with earlier lower bounds.

6.1 Idea of the reduction

All reductions start from SUBSET SUM [19]: given positive integers $a_1, \dots, a_n$ and B , decide whether $\sum_i a_i z_i = B$ for some $z \in \{0, 1\}^n$. In the indicator problem we build, the indicators choose the subset, and a chain of continuous variables adds up the chosen numbers.

Each number a_i gets a continuous *control* x_i with indicator z_i , and the objective contains the term $x_i^2 - 2A_i x_i + A_i^2 z_i$, where the target A_i is a scaled copy of a_i . If $z_i = 0$, then $x_i = 0$ and the term is zero. If $z_i = 1$, the term equals $(x_i - A_i)^2$, which is zero at $x_i = A_i$. Both choices therefore cost nothing as long as $x_i = A_i z_i$, so the indicators may select any subset and the controls record it.

A chain of continuous *states* $s_1, \dots, s_n$, with $s_0 = 0$, adds up the recorded numbers. For a small rational $\theta > 0$, the objective contains the squared *dynamic residuals* $(s_i - \theta s_{i-1} - \theta x_i)^2$. When all of them vanish, the states follow the recursion $s_i = \theta s_{i-1} + \theta x_i$, and unrolling it gives $s_n = \sum_i \theta^{n-i+1} x_i$. Each step shrinks the earlier part of the sum by the factor θ , and the targets undo this damping: with $A_i = a_i \theta^{-(n-i+1)}$ and $x_i = A_i z_i$, the last state is $s_n = \sum_i a_i z_i$. A final term $\theta^2 (s_n - B)^2$ compares this sum with B . On a YES instance, a solution of SUBSET SUM makes every square vanish.

The matrix is close to the identity when θ is small. Each variable appears with coefficient one in exactly one square, the control x_i in x_i^2 and the state s_i in its own residual, and every other square that contains it multiplies it by θ . Each residual involves only s_{i-1} , x_i , and s_i , which are at most two positions apart in the order $(x_1, s_1, \dots, x_n, s_n)$, so the matrix has bandwidth two. Because the coupling is small, the targets A_i , and with them the linear coefficients and the penalties, grow like powers of $1/\theta$.

The gap between YES and NO instances comes from integrality. On a NO instance, $B - \sum_i a_i z_i$ is a nonzero integer for every z , so its absolute value is at least one. Unrolling the recursion writes this integer as a fixed linear combination of the residuals: the control residuals $x_i - A_i z_i$, the dynamic residuals, and $s_n - B$. The residuals therefore cannot all be small, and the Cauchy–Schwarz inequality turns this into a lower bound on their cost in the objective. The bound is the reciprocal of a sum of squared weights, which are powers of θ , and this sum is bounded independently of n . Every NO solution therefore pays more than a constant $\Delta > 0$ that depends only on θ . Every continuous variable in (1) has its own indicator, so the states need indicators ζ_i as well. They get the small penalty $\Delta/(4n)$, so switching all of them on costs only $\Delta/4$, well below the gap.

The construction just described is case (1) of Theorem 6.1. It has a fixed matrix and a constant gap, but exponentially large coefficients and penalties. Case (2) rescales the variables and the objective of case (1) by a common factor, which bounds the data but can make the gap exponentially small. Case (3) moves the numbers a_i from the targets into the coefficients of the chain. The matrix then depends on the input, and every target and every penalty equals one. A state indicator now costs as much as a control indicator, and since the states are small, switching one off would cost little in the residuals and save a whole unit of penalty. Shifting all states by a constant D prevents this, because an inactive state must be zero, far from its required value near D .

6.2 Statement and proof

We prove hardness for the *rational threshold decision problem*: given a rational instance of (1) with $\xi = 0$ and a rational κ , decide whether the optimal value is at most κ . The objectives below contain additive constants, which let us write the residuals as squares. Subtracting such a constant from both the objective and κ gives an instance of (1).

Theorem 6.1. *Fix a rational $0 < \theta \leq 1/10$, and put*

$$\Delta = \frac{\theta^2(1 - \theta^2)}{1 + \theta^4}.$$

The rational threshold decision problem for indicator quadratic optimization is NP-complete under each of the following restrictions. In every case the matrix has bandwidth two, maximum graph degree at most four, treewidth and pathwidth two, and

$$(1 - 3\theta)I \preceq Q \preceq (1 + 3\theta + 2\theta^2)I. \quad (32)$$

- (1) *The matrix depends only on the dimension and θ , and all penalties are positive. The reduction produces instances with optimal value at most $\Delta/4$ from YES instances and greater than Δ from NO instances. Linear coefficients and penalties may have exponentially large magnitudes.*
- (2) *The same matrix family, with $|c_i| \leq 2$ and $0 < \lambda_i \leq 1$. The gap between YES and NO values may be exponentially small.*
- (3) *All penalties equal one and $|c_i| \leq 9$. The matrix depends on the input, through nonzero entries that may be exponentially small.*

In all cases the data have polynomial binary encoding length.

The three cases bound different numerical quantities:

Case	Matrix Q	$ c_i $	Penalties λ_i	Gap between YES and NO values
(1)	fixed	exponential	positive, exponential	constant ($\leq \Delta/4$ vs. $> \Delta$)
(2)	fixed	≤ 2	in $(0, 1]$	may be exponentially small
(3)	input-dependent	≤ 9	all equal to 1	may be exponentially small

Here “fixed” means that Q depends only on the dimension and θ , and “exponential” means possibly exponentially large in the dimension. Corollary 6.2 builds on case (1). In case (2) the optimal solutions are also bounded (see the proof), so only the gap is small. Deciding such an instance through the certificate (31) needs $n\sigma$ below the gap, so σ^{-1} , and with it the bound of Theorem 5.2, can be exponential. Case (3) shows that hardness does not need varied penalties, although no case combines an input-independent matrix with unit penalties. Because the reductions start from SUBSET SUM with binary-encoded numbers, they do not show strong NP-hardness.

Proof. Take a SUBSET SUM instance with positive integers $a_1, \dots, a_n, B$, where $n \geq 2$. As above, the variables x_i are controls and the variables s_i (in case (3), y_i) are states.

Case (1). Define $A_i = a_i\theta^{-(n-i+1)}$, $\eta = \Delta/(4n)$, and $s_0 = 0$. Use indicators z_i for x_i and ζ_i for s_i , and minimize

$$\Phi = \sum_i (x_i^2 - 2A_i x_i + A_i^2 z_i) + \sum_i (s_i - \theta s_{i-1} - \theta x_i)^2 + \theta^2 (s_n - B)^2 + \eta \sum_i \zeta_i. \quad (33)$$

The penalties are A_i^2 for z_i and η for ζ_i , and the quadratic part of Φ does not depend on the indicators. On indicator-feasible points $x_i = x_i z_i$ and $z_i = z_i^2$, so the first sum equals $\sum_i (x_i - A_i z_i)^2$.

If $\sum_i a_i z_i = B$, take $x_i = A_i z_i$, $s_i = \theta s_{i-1} + \theta x_i$, and every $\zeta_i = 1$. Then $s_n = \sum_i \theta^{n-i+1} A_i z_i = B$, all squares vanish, and $\Phi = n\eta = \Delta/4$.

For the converse, define the control residuals $e_i = x_i - A_i z_i$, the dynamic residuals $r_i = s_i - \theta s_{i-1} - \theta x_i$, and $u = s_n - B$. Unrolling the recursion for s_n shows that every feasible point satisfies

$$B - \sum_i a_i z_i = \sum_i \theta^{n-i+1} e_i + \sum_i \theta^{n-i} r_i - u.$$

On a NO instance the left side has absolute value at least one, and Cauchy–Schwarz, applied to $(e, r, \theta u)$, gives

$$\Phi \geq \|e\|^2 + \|r\|^2 + \theta^2 u^2 \geq \left[\theta^{-2} + (1 + \theta^2) \sum_{j=0}^{n-1} \theta^{2j} \right]^{-1} > \Delta.$$

The last inequality uses $\sum_{j=0}^{n-1} \theta^{2j} < 1/(1 - \theta^2)$ and $1/\Delta = \theta^{-2} + (1 + \theta^2)/(1 - \theta^2)$.

In the order $(x_1, s_1, \dots, x_n, s_n)$, every diagonal entry is $1 + \theta^2$, and the only off-diagonal entries are

$$Q_{x_i, s_i} = -\theta, \quad Q_{s_{i-1}, s_i} = -\theta, \quad Q_{s_{i-1}, x_i} = \theta^2 \quad (i \geq 2), \quad (34)$$

with only the first type present at $i = 1$. The bags $\{x_1, s_1\}, \{s_1, x_2, s_2\}, \dots, \{s_{n-1}, x_n, s_n\}$ give a width-two path decomposition, and since $n \geq 2$, the triangle $\{s_1, x_2, s_2\}$ makes the width exactly two. The maximum degree is at most four. Absolute off-diagonal row sums are at most $3\theta + \theta^2$, so Gershgorin’s theorem proves (32); in particular, the matrix is strictly diagonally dominant. For fixed rational θ , the powers defining A_i and all remaining data have polynomial encoding length, so the rational threshold $\Delta/4$ proves NP-hardness. The gap also rules out approximation: an estimate within absolute error $\Delta/4$ is at most $\Delta/2$ on YES instances and greater than $3\Delta/4$ on NO instances. Hence approximating the optimal value within absolute error $\Delta/4$ is also NP-hard.

Case (2). Let $M = 1 + B + \sum_i A_i$, substitute $(x, s) = M(\hat{x}, \hat{s})$ in (33), and divide the objective by M^2 . The matrix is unchanged, A_i and B become A_i/M and B/M , and the state penalty becomes η/M^2 . This gives the stated coefficient and penalty bounds and multiplies both thresholds by M^{-2} . Optimal solutions are also bounded. The all-zero point has value at most θ^2 , so at an optimum every control and dynamic residual has absolute value at most θ . Hence $|\hat{x}_i| \leq 1 + \theta$ and $|\hat{s}_i| \leq \theta(2 + \theta)/(1 - \theta) < 1$.

Case (3). We may assume $B \leq A := \sum_i a_i$, since otherwise the answer is NO. Set

$$b_i = \frac{a_i \theta^i}{A}, \quad T = \frac{B \theta^n}{A}, \quad D = 4,$$

so $0 < b_i \leq \theta$ and $0 < T \leq 1$. With $y_0 = D$, indicators z_i for x_i and ζ_i for y_i , and no constraints beyond the indicator constraints, minimize

$$G = \sum_i (x_i^2 - 2x_i + z_i) + \sum_i [y_i - \theta y_{i-1} - b_i x_i - D(1 - \theta)]^2 + \theta^2 (y_n - D - T)^2 + \sum_i \zeta_i. \quad (35)$$

The first dynamic residual is $y_1 - b_1 x_1 - D$. The shift D makes switching off a state indicator expensive, as shown below. For each z , let $s_0^z = 0$ and $s_i^z = \theta s_{i-1}^z + b_i z_i$; these are the values of $y_i - D$ when $x = z$ and every dynamic residual vanishes. Then $0 \leq s_i^z \leq \theta^i$, and $s_n^z = (\theta^n/A) \sum_i a_i z_i$ equals T exactly when $\sum_i a_i z_i = B$. Put $e = x - z$ and $d = y - D\mathbf{1} - s^z$, and let L be the lower shift matrix. On feasible points, $x_i^2 - 2x_i + z_i = e_i^2$, and the i th dynamic residual equals $d_i - \theta d_{i-1} - b_i e_i$ with $d_0 = 0$. The control and dynamic squares therefore satisfy

$$\|e\|^2 + \|(I - \theta L)d - \text{Diag}(b)e\|^2 \geq (1 - 3\theta)(\|e\|^2 + \|d\|^2). \quad (36)$$

Indeed, in this quadratic form a control row has diagonal $1+b_i^2$ and off-diagonal absolute sum at most $(1+\theta)b_i$. An internal state row has diagonal $1+\theta^2$ and row sum at most $2\theta+b_i+\theta b_{i+1} \leq 3\theta+\theta^2$, and the final row has diagonal one and row sum at most 2θ . Gershgorin's theorem gives (36).

If k state indicators vanish, the corresponding $d_i \leq -D$. Since $\theta \leq 1/10$ and $D = 4$,

$$G \geq n - k + (1 - 3\theta)D^2k \geq n + 10.2k.$$

The feasible choice $x = z = 0$, $y = D\mathbf{1}$, $\zeta = \mathbf{1}$ has value $n + \theta^2 T^2 < n + 1$. Consequently every optimum has all state indicators on. Its value is n plus a sum of squares, and it equals n exactly for a YES instance. Positive definiteness gives attainment, so every NO value is strictly greater than n . To quantify the gap, let $r_i = y_i - \theta y_{i-1} - b_i x_i - D(1 - \theta)$ and $u = y_n - D - T$. Using $e = x - z$ and $y_0 = D$, the recurrence gives

$$\frac{\theta^n}{A} \left(B - \sum_i a_i z_i \right) = \sum_i \frac{\theta^n a_i}{A} e_i + \sum_i \theta^{n-i} r_i - u.$$

At a NO optimum, Cauchy-Schwarz applied to $(e, r, \theta u)$ gives

$$G - n \geq \frac{\theta^{2n}/A^2}{\theta^{2n} \sum_i a_i^2/A^2 + \sum_{j=0}^{n-1} \theta^{2j} + \theta^{-2}} > 0.$$

The numerator can be exponentially small.

The quadratic matrix of (35) has diagonals $1+b_i^2$ on controls and $1+\theta^2$ on states. Its off-diagonal entries are $-b_i, -\theta, \theta b_i$ in the positions of (34). All are nonzero, and the row bounds above prove the graph claims and (32). Writing $h_1 = D$, $h_i = D(1 - \theta)$ for $i > 1$, the linear coefficients are $-2 + 2h_i b_i$ on controls, $-2h_i + 2\theta h_{i+1}$ on nonterminal states, and $-2h_n - 2\theta^2(D + T)$ on the last state. Their magnitudes are below nine. The constant is at most $16n + 1/4$, and all data have polynomial encoding length.

Membership in NP. For any rational positive-definite instance, a support J is a certificate. Its unique continuous optimum is $-Q_{JJ}^{-1} c_J/2$, and its value has polynomial rational encoding length by determinant bounds. Exact linear algebra verifies the threshold in polynomial time. Supports containing zero optimizing coordinates remain valid certificates. This proves membership in NP in all three cases. $\square$

All three cases come arbitrarily close to the identity. Dividing the objective of cases (1) and (2) by $1 + \theta^2$ gives a unit-diagonal matrix with $\|Q - I\|_2 \leq (3\theta + \theta^2)/(1 + \theta^2) \leq 4\theta$, and case (3) has $\|Q - I\|_2 \leq 3\theta + 2\theta^2$ by (32). At $Q = I$ the problem separates into scalar problems, so hardness appears arbitrarily close to a trivial case.

The instances also avoid a known polynomial case. Problem (1) is polynomial-time solvable when Q is a Stieltjes matrix, that is, has nonpositive off-diagonal entries, and the linear coefficients have one sign [2], as recalled by Liu et al. [27]. The linear coefficients in all three cases are nonpositive, so it matters that the matrices are not Stieltjes. Changing the signs of some continuous variables turns an instance into an equivalent one, but it cannot make these matrices Stieltjes. Each triangle of the support graph has a positive product of off-diagonal entries, sign changes preserve this product, and in a Stieltjes matrix the product would be negative.

Although we call the variables controls and states, the instances lie outside the multi-period model of Lee, Gómez, and Atamtürk [24]. There the dynamics are exact constraints and the quadratic cost involves only the states; here the dynamics are only penalized, and every control has its own square. Even with all state indicators on, minimizing over the states leaves the control matrix $I + \alpha w w^T$, where $\alpha > 0$, $w_i = \theta^{n-i+1}$ in cases (1) and (2), and $w_i = \theta^{n-i} b_i$ in case (3). Its inverse has no zero entries, so for $n \geq 3$ it is not tridiagonal.

6.3 Consequence for perturbed penalties

Theorem 6.1 concerns the unperturbed problem. Corollary 6.2 below fixes the matrix family of case (1), hence H and μ , and the perturbation scale σ ; it does not involve R ; and it scales only the linear data c and λ .

Corollary 6.2. *Fix a rational $\sigma > 0$, and let each ξ_i be drawn independently and uniformly from the perturbation grid (11), with N as in Theorem 5.2. Suppose an algorithm returns the exact optimal value of (1) for every draw, and there is a polynomial p such that, on every instance of encoding length ℓ , its expected bit complexity over the draw is at most $p(\ell)$. Then $\text{NP} \subseteq \text{ZPP}$. This holds even if the algorithm is required to work only for the fixed matrix family of Theorem 6.1(1).*

The proof is a scaling argument. Multiplying the linear coefficients of a case-(1) instance by γ and its penalties by γ^2 multiplies the gap between YES and NO values by γ^2 , while the perturbation changes every value by at most $2n\sigma$, independently of γ . For large γ the exact perturbed value therefore still decides the SUBSET SUM instance. Beier and Vöcking use the same scaling to show that polynomial smoothed complexity implies a randomized pseudopolynomial algorithm [3, conference version, Theorem 3]; see also [29, Theorem 6.2].

Proof. Start from a case-(1) instance. It has $n_{\text{tot}} = 2n$ indicators, and its smallest penalty is $\eta = \Delta/(4n)$. Choose an integer $\gamma \geq 1$ with $3\gamma^2\Delta/4 > 2n_{\text{tot}}\sigma$ and $\gamma^2\Delta/(4n) > \sigma$. Such a γ has polynomial encoding length and is computable by rational comparisons. Let c_0 denote the additive constant in the expanded objective (33). Replace (c, λ, c_0) by $(\gamma c, \gamma^2 \lambda, \gamma^2 c_0)$, keeping Q . Substituting $\gamma x'$ for the vector x of all continuous variables shows that every feasible value is multiplied by γ^2 . A perturbation with $|\xi_i| \leq \sigma$ changes the optimum by at most $n_{\text{tot}}\sigma$. The perturbed YES value is therefore at most $\gamma^2\Delta/4 + n_{\text{tot}}\sigma$, whereas the NO value exceeds $\gamma^2\Delta - n_{\text{tot}}\sigma$. These ranges are disjoint for every draw, and all perturbed penalties remain positive. Sample the perturbation, run the solver, and compare its exact answer with the midpoint of these two bounds. The assumed expected bound gives a zero-error expected-polynomial algorithm for SUBSET SUM. $\square$

The scaled case-(1) instances have exponentially large C , so the bound of Theorem 5.2 is exponential for them. The argument scales the penalties together with the linear coefficients, so it gives no obstruction when the penalties are bounded.

7 Large exact messages from fixed local data

Section 6 shows that exact optimization can be hard. The size of a message is a different obstacle, and it appears even when optimization is easy. A message is the pointwise minimum of its branches, and an exact dictionary must keep every branch that is the unique minimum at some boundary value.

Section 7.1 builds the instances of Theorem 7.2 from identical stages, and Section 7.2 proves that one of their messages needs 2^n branches in every exact dictionary, while w , C , H , μ^{-1} , and R stay bounded and the input length is polynomial in n . Hence, without perturbation, the dictionary size has no bound polynomial in the input length and these quantities. Section 7.3 treats approximation within a uniform error ε (Theorem 7.3).

7.1 A chain that writes the support into its final state

Fix a rational $0 < \theta \leq 1/10$ and define, for $t \in [0, 1]$,

$$V_n(t) = \min \left\{ \sum_{i=1}^n (x_i^2 - 2x_i + z_i) + \sum_{i=1}^n (s_i - \theta s_{i-1} - \theta x_i)^2 : \begin{array}{l} x_i(1 - z_i) = 0, \quad z_i \in \{0, 1\}, \\ s_0 = 0, \quad s_n = t \end{array} \right\}. \quad (37)$$

The controls x_i have indicators z_i , and the states s_i are free. Each stage penalizes deviation from the dynamics $s_i = \theta s_{i-1} + \theta x_i$, and every stage has the same coefficients. On indicator-feasible points, $x_i^2 - 2x_i + z_i = (x_i - z_i)^2$, so an inactive control is zero and an active control costs nothing exactly when it equals one. The terminal equality $s_n = t$ makes V_n a conditional value function of the boundary value t ; it is not a constraint of the global problem. We identify a support with its indicator vector $z \in \{0, 1\}^n$. As in Section 2.4, the branch of z is the minimum over the continuous variables with the support fixed to z . Unlike the message branches (8), these branches keep the terms that involve only t , which makes them convex rather than concave in t . Up to a shift of t , the message constructed in the proof of Theorem 7.2 differs from V_n by one fixed polynomial, which does not change the number of formulas needed.

For a fixed support z , the choice $x = z$ with exact dynamics costs nothing and drives the state to the center $p_z = \sum_{i=1}^n \theta^{n-i+1} z_i$. Each stage multiplies the state by θ , so the control of stage i reaches the end of the chain with weight θ^{n-i+1} . The center is therefore the number with digits $z_n, z_{n-1}, \dots, z_1$ in base $1/\theta$. For $\theta = 1/10$ these are ordinary decimal digits: with $n = 3$, the support $z = (1, 0, 1)$ has center 0.101, and $z = (0, 1, 1)$ has center 0.110. Reaching another terminal value t costs $(t - p_z)^2$ divided by a factor $D_z \in [1, 2)$ (Lemma 7.1). Because θ is small, distinct digit strings give centers at least θ^n apart, so each support is the unique best one near its own center.

This is how fixed local data produce exponentially many branches. No coefficient is large, and none changes from stage to stage. What grows with n is the number of digits of t that matter: to tell all 2^n supports apart, one must know t to within about θ^n , and a single continuous boundary value can carry that many digits.

Lemma 7.1. *Let $W_n = \sum_{j=0}^{n-1} \theta^{2j}$. The branch of support z in (37) is*

$$q_z(t) = \frac{(t - p_z)^2}{D_z}, \quad p_z = \sum_{i=1}^n \theta^{n-i+1} z_i, \quad D_z = W_n + \sum_{i=1}^n \theta^{2(n-i+1)} z_i. \quad (38)$$

Thus $V_n = \min_z q_z$.

Proof. Fix z and put $e_i = x_i - z_i$ and $r_i = s_i - \theta s_{i-1} - \theta x_i$. Inactive controls force $e_i = 0$, and the objective is $\sum_i (e_i^2 + r_i^2)$. Unrolling the dynamics from $s_0 = 0$ to $s_n = t$ leaves the single linear constraint

$$t - p_z = \sum_i \theta^{n-i+1} e_i + \sum_i \theta^{n-i} r_i.$$

Conversely, any residuals that satisfy it, with $e_i = 0$ for inactive i , determine feasible controls and states. The free residuals are e_i for active i and every r_i , and their coefficients in this constraint have squared norm D_z . Cauchy–Schwarz therefore gives the minimum $(t - p_z)^2/D_z$, attained at $e_i = (t - p_z)\theta^{n-i+1}z_i/D_z$ and $r_i = (t - p_z)\theta^{n-i}/D_z$. $\square$

7.2 All branches are needed

Theorem 7.2. *Every finite exact representation of V_n on $[0, 1]$ as a minimum of polynomials, or as a piecewise polynomial function, uses at least 2^n distinct polynomial formulas. The same holds for a*

scalar conditional message, on any box $[-R, R]$ with $R \geq 5$, of an unperturbed instance of (1) with unit penalties on every variable, coefficients from a fixed finite alphabet of rationals depending only on θ , and the graph and spectral bounds of Theorem 6.1 (width at most two for all n , and exactly two when $n \geq 2$).

Proof. Each q_z is the unique minimum on an interval around p_z , so every finite representation must contain q_z itself. For the second claim, we give the free states unit-penalty indicators that no minimizer switches off.

Put $\kappa_\theta = (1 + \theta^2)/(1 - \theta^2) < 2$. Every D_z lies in $[1, \kappa_\theta]$, and all centers lie in $[0, \theta/(1 - \theta)]$. Distinct centers are at least θ^n apart. Indeed, the bit z_i carries the power θ^{n-i+1} . Let k be the smallest exponent at which two supports differ, that is, the leading digit in which their centers differ. If $k < n$, their centers differ by at least $\theta^k(1 - 2\theta)/(1 - \theta) \geq \theta^n$, using $\theta \leq 1/10$; if $k = n$, they differ by exactly θ^n . On $|t - p_z| \leq \theta^n/3$, intersected with $[0, 1]$, we have

$$q_z(t) \leq \theta^{2n}/9, \quad q_{z'}(t) \geq 4\theta^{2n}/(9\kappa_\theta) > q_z(t) \quad (z' \neq z).$$

So q_z is the unique minimum, and equals V_n , on a nonempty open interval. If finitely many polynomials represent V_n there, one of them agrees with q_z at infinitely many points and hence identically, because a nonzero polynomial has only finitely many roots. This argument applies both to a minimum and to a piecewise representation, and to polynomials of any degree. Distinct centers give distinct polynomials.

It remains to realize V_n as a message of (1). Before s_n is fixed, the quadratic matrix in (37), in the order $(x_1, s_1, \dots, x_n, s_n)$, has the edges (34), diagonal entries $1 + \theta^2$ for controls and nonterminal states, and final diagonal entry one. The graph is that of Theorem 6.1, and the same Gershgorin row bounds prove (32); the final row has off-diagonal sum 2θ .

Problem (1) attaches an indicator to every variable, while the states in (37) are free. We therefore shift the states by four and give each one a unit-penalty indicator. The shift moves the reference trajectories away from zero, so switching a state indicator off, which forces that state to zero, leaves a large residual. Let $y_0 = 4$, attach an indicator ζ_i to y_i , and replace the dynamic terms by

$$\sum_i [y_i - \theta y_{i-1} - \theta x_i - 4(1 - \theta)]^2 + \sum_i \zeta_i, \quad y_n = 4 + t. \quad (39)$$

With $y = 4\mathbf{1} + s$ and every $\zeta_i = 1$, the lifted objective is the objective of (37) plus n . The separator is $S = \{y_n\}$; the lower bound already holds on $y_n \in [4, 5] \subseteq [-R, R]$, which corresponds to $t \in [0, 1]$.

No minimizer switches a state indicator off. To see this, fix a control support z with reference states $s_0^z = 0$ and $s_i^z = \theta s_{i-1}^z + \theta z_i \geq 0$. With $e = x - z$ and $d = y - 4\mathbf{1} - s^z$, the control and dynamic squares are the left side of (36) with $b_i = \theta$, so their sum is at least $(1 - 3\theta)(\|e\|^2 + \|d\|^2)$. An indicator $\zeta_i = 0$ forces $y_i = 0$, hence $d_i \leq -4$. If k state indicators are off, the objective is at least $n - k + 16(1 - 3\theta)k \geq n + 10.2k$. The choice with all state indicators on, $x = z = 0$, $y_i = 4$ for $i < n$, and $y_n = 4 + t$ costs $n + t^2 \leq n + 1$. Hence every minimizer has all state indicators on, and the conditional value is $n + V_n(t)$.

The message convention (6) omits the terminal penalty, which leaves $n - 1 + V_n(t)$. It also omits the terms involving only y_n , and writing the lifted objective in the form (1) drops an additive constant, which is at most $16n$. So the message differs from $n - 1 + V_n$ by a fixed polynomial. Subtracting one fixed polynomial from every formula does not change the number of indispensable formulas. The omitted terms include y_n^2 with coefficient one, while each q_z , as a function of y_n , has leading coefficient $1/D_z \leq 1$; this is why the convex functions q_z become concave branches of the message. The expanded linear coefficients have magnitude at most eight. The quadratic and linear coefficients and the unit penalties belong to a finite alphabet depending only on θ . $\square$

The theorem counts explicit polynomial formulas. It is not a lower bound for implicit representations, such as the optimization problem (37) that defines V_n , nor for optimization, because the global problem is trivial. Without the terminal condition, every control support has the zero-residual trajectory $x = z$, so the optimal value of (37) without $s_n = t$ is zero, and that of the lifted problem is n . The example is thus an extreme case of near ties between supports. Margin conditions, such as that of [5, Definition 5], are designed to limit near ties of this kind. The perturbation in Theorem 5.2 shifts the branch of each support A by $\sum_{i \in A} \xi_i$, and these random shifts limit near ties in expectation (Lemma 4.1); the graph and the conditioning alone do not.

7.3 Matching bounds for pruning branches

If a uniform error ε is allowed, a natural approach keeps only some branches q_z and discards the rest. Let $N_n(\varepsilon)$ be the smallest number of branches q_z whose minimum $\hat{V}$ satisfies $0 \leq \hat{V} - V_n \leq \varepsilon$ on $[0, 1]$, and set $N_\theta(\varepsilon) = \sup_{n \geq 1} N_n(\varepsilon)$. This supremum is finite, and Theorem 7.3 gives its order of growth as $\varepsilon \downarrow 0$.

Theorem 7.3. *As $\varepsilon \downarrow 0$,*

$$N_\theta(\varepsilon) = \Theta_\theta(\varepsilon^{-\alpha_\theta}), \quad \alpha_\theta = \frac{\log 2}{2 \log(1/\theta)}.$$

The lower bound concerns only minima of branches q_z .

Proof idea. Keeping the m most significant digits of the centers (the bits $z_{n-m+1}, \dots, z_n$) locates every center to within about θ^m , which gives error of order θ^{2m} with about 2^m branches. Conversely, V_n vanishes at 2^m centers spaced at least θ^m apart, and once ε is small compared with θ^{2m} , each of these zeros needs its own branch. Each additional digit thus doubles the number of branches and multiplies the error by about θ^2 , which gives the exponent α_θ ; for $\theta = 1/10$, $\alpha_\theta \approx 0.15$.

Proof. For the upper bound, keep the last m bits $z_{n-m+1}, \dots, z_n$ of a support, where $0 \leq m \leq n$; they carry the powers $\theta, \dots, \theta^m$. Let

$$\delta = \sum_{j=m+1}^n \theta^j \leq \frac{\theta^{m+1}}{1-\theta}, \quad \beta = \sum_{j=m+1}^n \theta^{2j} \leq \frac{\theta^{2m+2}}{1-\theta^2}.$$

For each choice of the kept bits, retain the two supports whose omitted bits are all zero or all one. These are at most 2^{m+1} of the branches q_z , each with its own denominator. For an optimal support z at t , the two retained centers p_0, p_1 with the same kept bits bracket p_z and are δ apart. The one closer to t has squared distance at most $(t - p_z)^2 + \delta^2/4$ from t : outside $[p_0, p_1]$ it is no farther than p_z , and inside the interval it is within $\delta/2$. Its denominator differs from D_z by at most β ; all denominators are at least one and $(t - p_z)^2 \leq 1$. Consequently

$$0 \leq \hat{V} - V_n \leq \delta^2/4 + \beta \leq A_\theta \theta^{2m+2}, \quad A_\theta = \frac{1}{4(1-\theta)^2} + \frac{1}{1-\theta^2}. \quad (40)$$

Taking $m = \lceil \log(A_\theta \theta^2/\varepsilon)/(2 \log(1/\theta)) \rceil$, or retaining every support if $n \leq m$, proves the upper bound $4(A_\theta \theta^2/\varepsilon)^{\alpha_\theta}$ for sufficiently small ε .

For the lower bound take $n \geq m \geq 1$. The 2^m supports whose first $n - m$ bits are zero have centers separated by at least θ^m , by the separation argument in the proof of Theorem 7.2, and V_n vanishes at each of these centers. Since $\hat{V} \leq V_n + \varepsilon$, some retained branch is at most ε at each such center, so its own center lies within $\sqrt{\kappa_\theta \varepsilon}$. If $\varepsilon < \theta^{2m}/(4\kappa_\theta)$, a retained branch can serve at most one of the 2^m centers. Choose $m = \lfloor \log(1/(8\kappa_\theta \varepsilon))/(2 \log(1/\theta)) \rfloor$. Then $N_\theta(\varepsilon) \geq \frac{1}{2}(8\kappa_\theta \varepsilon)^{-\alpha_\theta}$. $\square$

Section 3 compares Theorems 7.2 and 7.3 with known value functions that have exponentially many pieces [21, 28] and with known pruning rates [17].

8 Discussion

8.1 Practical relevance

The bound of Theorem 5.2 is far from a practical running time. In (29), the net size J_* and the grid factor $(K + 2)^{w+1}$ are $(w + 1)$ st powers of quantities that grow polynomially with n , C , H , μ^{-1} , R , and σ^{-1} . To see the scale, take $n = 100$, $w = 2$, $H = 2$, $\mu = C = R = 1$, and $\sigma = 1/10$. Then (28) and (25) give $J_* > 5 \cdot 10^{20}$ and $K > 6 \cdot 10^4$, so $J_*(K + 2)^{w+1} > 10^{35}$ even before the factors b and $P_w(\ell)$ are included. Moreover, every oracle call runs a full grid dynamic program, and each message uses at most $J_*(1 + en)$ oracle calls in expectation (Step 4 of the proof of Theorem 5.2). These factors come from bounds chosen for the analysis: one Lipschitz constant and one grid serve all supports, and the net ignores where the optimal support changes. We have not tried to reduce them. An exact-rational reference implementation, available from the authors, replaces X and L by bounds specific to each message. We used it only to check correctness on small instances, including supports that are optimal only at a tie point.

Nor does Theorem 5.2 imply a speedup over existing exact methods, such as the parametric dynamic programs of [5, 6] discussed in Section 3. In the synthetic banded experiments of [5, Section 7.2, Table 1], with bandwidth two or four and n up to 2,000, the reported average number of quadratic pieces retained after pruning (branches, in our terminology) is between 23 and 1,139. Exponential messages such as that of Theorem 7.2 therefore need not arise on such data. The guarantee of Theorem 5.2 matters most for instances with many near ties, like the example of that theorem.

The grid oracle of Lemma 5.1 solves the problem without side constraints. A cardinality bound on the support can be added at a polynomial cost, by also recording the number of active vertices in the dynamic-program tables. Nonlinear constraints on x would need a different oracle. Theorem 4.3 itself applies to any support family for which a restricted additive oracle is available.

8.2 Possible practical uses

Two parts of the analysis may still be useful in practice, although we have not tested either of them.

The first is the penalty perturbation itself, used inside a pruned exact dynamic program. The exact algorithm of [5] prunes branches that are never optimal on a bounded region of boundary values. Its complexity bound rests on a deterministic margin condition [5, Definition 5 and Theorem 1]. At each bag B_u , give every support of the local subproblem the optimal cost of that subproblem with the bag variables set to zero, and group the supports into classes by their indicator values on the vertices within a fixed graph distance of B_u . The η -optimal set of a class consists of its supports whose cost is within η of the least cost in the class. The condition requires a single k that bounds the size of every η -optimal set, over all bags and classes. Under our perturbation, Lemma 4.1 bounds the expected size of each such set by e when $\eta \leq \sigma/(2n)$. An expected bound for each set is weaker than their uniform bound, so this gives no running-time guarantee for their algorithm. Perturbing the penalties may still shorten the lists that their algorithm keeps on instances with many near ties. The loss in accuracy is bounded in advance: from an exact solution of the perturbed problem, (30) gives lower and upper bounds on the original optimum that differ by at most $n\sigma$ for every draw.

The second is certified enumeration with oracles other than the grid dynamic program. Lemma 4.2 works with any restricted additive oracle in the sense of (16). From such an oracle it builds a list E

that contains every support within δ of the optimum and only supports within $\delta + 2\varepsilon$. It makes at most $1 + n|E|$ oracle calls, so its cost grows with the number of near-optimal supports, which can be exponential without perturbation.

Approximate decision diagrams, described in Section 3, can serve as such oracles on the structures they cover. In the truncated diagrams of [18] and the neighborhood-merged diagrams of [11], each support is encoded by a path whose length differs from the cost of that support by at most a bound that is the same for all supports. For [18] the bound follows by summing [18, Proposition 7] along the path; for [11] it is [11, Proposition 9], and it is proportional to the merging tolerance. If the truncation depth or the merging tolerance is chosen so that this bound is at most $\varepsilon/2$, a shortest path through the arcs consistent with the fixed bits, returned with the exact cost of its support, is a restricted additive oracle with accuracy ε . The diagrams also need not be rebuilt at each net point. As noted in Section 3, they are built from the matrix alone, and only their arc lengths change with the linear coefficients and penalties. In (6) the boundary value t enters only through the linear coefficients $c_I + 2Q_{Ist}$. Hence one diagram for Q_{II} , with its accuracy set for the largest linear coefficients on the box, serves every net point of a message.

8.3 Open questions

1. *Computational study.* How large are the dictionaries of perturbed instances in practice, and how do they compare with the pruned lists of exact parametric dynamic programs [5, 6] on the same data?
2. *Avoiding the parameter net.* The net covers the whole box uniformly, and its size J_* is a large factor in (29). Is there an adaptive search over boundary values with a provable expected bound, for example one that refines the net only where the enumerated lists change?
3. *Recursive composition.* Our algorithm computes each message directly from its own subproblem. This avoids products of child dictionaries but repeats work. Can child dictionaries be combined bottom-up, with an expected bound on the intermediate lists, without additional assumptions on Q ?
4. *Deterministic conditions.* Theorem 5.2 bounds the expected dictionary size over perturbation draws for every instance, but it does not say which unperturbed instances have small messages. Which checkable conditions on unperturbed data, other than the margin and volume-growth conditions of [5], guarantee polynomial message size at fixed treewidth?
5. *Finer representation lower bounds.* The star argument of Appendix A concerns the inverse-principal polytope including its inverse entries. Are there exponential lower bounds for formulations that project out the inverse entries, such as extended formulations of the closed convex hull of the epigraph?

A Representation size on stars

This appendix proves the lower bound on stars that Section 3 describes for the inverse-principal polytope of Wei et al. [30]. Remark A.3 checks the corresponding observation of Choi et al. [11, Section 7.2] for their decision diagrams.

Formulation size is measured with extended formulations, because auxiliary variables can make a formulation much smaller. An *extended formulation*, or *lift*, describes a convex set as the projection of a higher-dimensional convex set that is easier to write down. For example, the unit ball of the

ℓ_1 norm in $\mathbb{R}^n$ has 2^n facets, but it is the projection onto x of $\{(x, y) : -y \leq x \leq y, \mathbf{1}^T y \leq 1\}$, which needs only $2n + 1$ inequalities. The size of the smallest lift, called the extension complexity, therefore measures how hard a polytope is to formulate.

The basic example of a polytope without small lifts is the *correlation polytope* $\text{COR}(m) = \text{conv}\{aa^T : a \in \{0, 1\}^m\}$, also known as the Boolean quadric polytope [16, Section 3.2]. Every linear lift of $\text{COR}(m)$ has $2^{\Omega(m)}$ inequalities [16, Theorem 7]; the proof of Corollary A.2 recalls the corresponding bounds for second-order-cone and semidefinite lifts. These bounds pass to every polytope with a face affinely isomorphic to $\text{COR}(m)$, because restricting a lift of the polytope to the hyperplane that cuts out the face gives a lift of the face that is no larger. Fiorini et al. derive their bounds for stable-set and travelling-salesman polytopes in the same way, from faces that project onto $\text{COR}(m)$ [16, Lemmas 8, 9, and 11].

Formally, a lift, or conic extended formulation, of a polytope P over a closed convex cone K writes $P = \pi(K \cap \mathcal{A})$ for an affine subspace $\mathcal{A}$ and an affine map π ; its size is measured by the cone K . For a linear lift, K is a nonnegative orthant $\mathbb{R}_+^r$, and r counts the inequalities. If an affine function g is nonnegative on P , then $\mathcal{A}' = \mathcal{A} \cap \{y : g(\pi(y)) = 0\}$ satisfies $\pi(K \cap \mathcal{A}') = P \cap \{g = 0\}$, which gives a lift of this face over the same cone. Composing with an affine isomorphism onto $\text{COR}(m)$ gives a lift of $\text{COR}(m)$. Every lower bound on lifts of the correlation polytope therefore applies to any polytope with such a face.

For a positive-definite matrix Q with index set V , the *inverse-principal polytope* is

$$P_Q = \text{conv}\{(\mathbf{1}_S, W^S) : S \subseteq V\}, \quad W_{SS}^S = Q_{SS}^{-1}, \quad W_{ij}^S = 0 \text{ otherwise},$$

with $W^\emptyset = 0$. Each point $(\mathbf{1}_S, W^S)$, which we call an atom, pairs the indicator vector of S with the inverse of the principal submatrix Q_{SS} , padded with zeros. This polytope yields a convex-hull description of the epigraph of $x^T Q x$ with indicators: Wei et al. [30, Section 3.1 and Theorem 1] show that the closed convex hull of $\{(x, z, t) : t \geq x^T Q x, x_i(1 - z_i) = 0, z \in \{0, 1\}^V\}$ is the set of points for which some W satisfies $(z, W) \in P_Q$ and $\begin{pmatrix} W & x \\ x^T & t \end{pmatrix} \succeq 0$. A small extended formulation of P_Q therefore gives a small extended formulation of this hull. Liu et al. [27] study Stieltjes matrices, the positive-definite matrices with nonpositive off-diagonal entries. They prove hardness of optimizing over a related inverse-graph closure for a diagonal-minus-rank-one matrix [27, Section 4.3, Proposition 6]. We show that P_Q itself has no small exact extended formulation, even for a sparse, uniformly conditioned star.

Theorem A.1. *For $m \geq 1$, let $b = 1/(2m)$ and*

$$Q = \begin{pmatrix} 1 & b\mathbf{1}^T \\ b\mathbf{1} & I_{2m} \end{pmatrix},$$

with rows and columns indexed by the root 0 and the leaves $1, \dots, 2m$. Then Q has a star support graph, rational entries of $O(\log(m + 1))$ bits, and condition number below six. The polytope P_Q has an exposed face affinely isomorphic to the correlation polytope $\text{COR}(m) = \text{conv}\{aa^T : a \in \{0, 1\}^m\}$. The same holds for the Stieltjes matrix obtained by changing the sign of the root coordinate.

The construction works because, when the root is selected, inverting Q_{SS} couples any two selected leaves through the root: the inverse entry between distinct leaves i and j is then a multiple of $z_i z_j$, like the entry $a_i a_j$ of aa^T . The multiple depends on how many leaves are selected. To keep it fixed, the proof pairs leaf i with leaf $m + i$ and uses an affine function that vanishes exactly at the atoms selecting the root and one leaf of each pair. These atoms select m leaves, so on them the inverse entry between distinct leaves $i, j \leq m$ is a fixed multiple of $z_i z_j$, as in $\text{COR}(m)$.

Proof. The eigenvalues of Q are one and $1 \pm 1/\sqrt{2m}$, so its condition number is at most $(1 + 1/\sqrt{2})/(1 - 1/\sqrt{2}) = 3 + 2\sqrt{2} < 6$.

We first compute the atoms. Write L for the leaves, $z \in \{0, 1\}^{2m}$ for the leaf indicators of an atom, and $k = \mathbf{1}^T z$. If the root is not selected, the leaf block of W is $\text{Diag}(z)$. If the root is selected, put $\omega = (1 - b^2 k)^{-1}$, which is finite because $b^2 k \leq 1/(2m) < 1$. Block inversion gives

$$W_{00} = \omega, \quad W_{0L} = -b\omega z^T, \quad W_{LL} = \text{Diag}(z) + b^2 \omega z z^T. \quad (41)$$

Next we expose the face. Consider the affine function

$$g(z_0, z, W) = (m+1)(1 - z_0) + m - \mathbf{1}^T z + b^{-2} \sum_{i=1}^m W_{i, m+i}.$$

At an atom without the root, $g = 2m + 1 - k \geq 1$. At an atom with the root, let p be the number of pairs $\{i, m+i\}$ with both leaves selected. Then $b^{-2} \sum_i W_{i, m+i} = \omega p$ and $k \leq m + p$, so $g = m - k + \omega p \geq (\omega - 1)p \geq 0$. If $p > 0$, then $k \geq 2$ and $\omega > 1$, so $g > 0$. Hence $g \geq 0$ on P_Q , and an atom has $g = 0$ exactly when the root and exactly one leaf of each pair are selected, that is, when $z = (a, \mathbf{1} - a)$ for some $a \in \{0, 1\}^m$. The hyperplane $g = 0$ therefore exposes the face $\mathcal{F}$ of P_Q spanned by these atoms.

Finally we identify $\mathcal{F}$ with $\text{COR}(m)$. On $\mathcal{F}$, $z_0 = 1$ and $k = m$, so $\omega = (1 - mb^2)^{-1}$ is constant. Define the affine map ϕ by $X_{ii} = z_i$ and $X_{ij} = W_{ij}/(b^2 \omega)$ for distinct $i, j \in \{1, \dots, m\}$. By (41), ϕ sends the atom with $z = (a, \mathbf{1} - a)$ to aa^T , so $\phi(\mathcal{F}) = \text{COR}(m)$. For the inverse, let $a = \text{diag}(X)$, $z = (a, \mathbf{1} - a)$, and

$$R(X) = \begin{pmatrix} X & a\mathbf{1}^T - X \\ \mathbf{1}a^T - X & \mathbf{1}\mathbf{1}^T - a\mathbf{1}^T - \mathbf{1}a^T + X \end{pmatrix}.$$

Let $\psi(X)$ be the point with $z_0 = 1$, leaf indicators z , $W_{00} = \omega$, $W_{0L} = -b\omega z^T$, and $W_{LL} = \text{Diag}(z) + b^2 \omega R(X)$. The map ψ is affine, and $R(aa^T) = zz^T$, so $\psi(aa^T)$ is the atom (41). Thus $\psi \circ \phi$ is the identity at the atoms of $\mathcal{F}$, and $\phi \circ \psi$ at the points aa^T . Both maps are affine, so they are the identity on $\mathcal{F}$ and on $\text{COR}(m)$.

For the sign change, let $D = \text{Diag}(-1, 1, \dots, 1)$. Every padded principal inverse of DQD equals $DW^S D$, so the invertible linear map $(z, W) \mapsto (z, DWD)$ maps P_Q onto P_{DQD} . $\square$

Corollary A.2. *Let Q be either matrix in Theorem A.1. An exact lift of P_Q needs exponentially many nonnegative scalar factors if it is a linear program, exponential total cone dimension if it uses second-order cones, and exponentially many positive-semidefinite blocks if every block has order at most a fixed d . Scalar inequalities count as blocks of order one. A semidefinite lift with blocks of arbitrary order needs total block order $2^{\Omega(m^{2/13})}$.*

Proof. By the restriction argument before Theorem A.1, it suffices to bound lifts of $\text{COR}(m)$ over the same cones. For fixed d , Fawzi and Parrilo [15, Theorem 1] show that the number of blocks of order d grows exponentially in m ; blocks of smaller order embed in blocks of order d . A linear program is the case $d = 1$, where the bound is due to Fiorini et al. [16, Theorem 7]. For $d = 2$, at least $7^{-1/2}(9/7)^{m/2}$ blocks are needed. A Lorentz cone of dimension $k \geq 3$ has a lift with $k - 2$ three-dimensional Lorentz cones (a standard construction, recalled in [15, p. 3]), each linearly isomorphic to the positive-semidefinite cone of order two. Lorentz cones of dimensions one and two are linearly isomorphic to products of one or two scalar cones. A second-order-cone lift of total dimension D thus gives a lift with at most D blocks of order at most two. For blocks of arbitrary order, place them in one block-diagonal matrix whose order is their total order, and apply Lee, Raghavendra, and Steurer [23, Theorems 1.1 and 5.4]. $\square$

The lower bound concerns P_Q together with all its inverse entries. The obstruction lies in the leaf–leaf entries of W , which belong to pairs of vertices that are not adjacent in the star, and it disappears when they are projected out. On $\mathcal{F}$, the indicators, the diagonal of W , and the entries W_{0i} on star edges are affine functions of $a = (z_1, \dots, z_m)$, so their projection is affinely isomorphic to the cube $[0, 1]^m$. The argument therefore gives no lower bound for formulations of the indicator quadratic epigraph that eliminate W .

Remark A.3 (Projected-row decision diagrams on stars). The decision diagrams of Choi et al. [11] fix the indicators one at a time in a chosen variable order. Each node of a layer is a state that summarizes how the indicators fixed so far affect the rest of the problem, and partial assignments with the same state share a node; the size of a diagram is its number of nodes. Choi et al. [11, Section 7.2 and Corollary 1] bound the number of nodes of their exact decision diagrams on a tree with k leaves by a polynomial of degree $k + 1$ in the number of variables, for a variable order in which every vertex of the tree precedes its parent. They note that on stars the resulting representation becomes exponential in size, although tree problems are solvable in polynomial time. Their states store the future rows of a factor F with $Q = FF^T$, projected orthogonally away from the rows of the selected past variables [11, Definition 7 and Lemma 2]; their Section EC.2.2 also allows normalizing each stored row.

A direct check shows that these diagrams are exponential on stars for every variable order. Let Q have unit diagonal, root 0, leaves $1, \dots, n$, entries $Q_{0i} = b_i \neq 0$, and $\sum_i b_i^2 < 1$ (for example, $b_i = 1/(2n)$, so that $\|Q - I\|_2 = 1/(2\sqrt{n})$). With unconstrained supports, the layer just before the last leaf j in the order has at least 2^{n-1} distinct states, with or without row normalization. Indeed, the rows $f_0, \dots, f_n$ of F are linearly independent, the leaf rows are orthonormal, and $\langle f_0, f_i \rangle = b_i$. Let S be the set of selected leaves other than j . If the root follows j , the state contains the root row $r_0(S) = f_0 - \sum_{i \in S} b_i f_i$. If the root precedes j and is selected, the state is the row $r_j(S) = f_j - (b_j/d_S) r_0(S)$ of j , where $d_S = \|r_0(S)\|^2 = 1 - \sum_{i \in S} b_i^2 > 0$. In the basis $f_0, \dots, f_n$, each row has one coefficient equal to one, and its coefficients on leaves $i \neq j$ are nonzero exactly for $i \in S$. Different subsets thus give nonproportional rows, which row scaling cannot merge. This confirms their observation and is not a contribution of this paper.

References

- [1] David Applegate, Aaron Archer, David S. Johnson, Evdokia Nikolova, Mikkel Thorup, and Ger Yang. Wireless coverage prediction via parametric shortest paths. In *Proceedings of the Eighteenth ACM International Symposium on Mobile Ad Hoc Networking and Computing*, pages 221–230, 2018. doi: 10.1145/3209582.3209605. URL <https://arxiv.org/abs/1805.06420v1>. Inspected preprint: arXiv:1805.06420v1.
- [2] Alper Atamtürk and Andrés Gómez. Strong formulations for quadratic optimization with M-matrices and indicator variables. *Mathematical Programming*, 170:141–176, 2018. doi: 10.1007/s10107-018-1301-5.
- [3] René Beier and Berthold Vöcking. Typical properties of winners and losers in discrete optimization. *SIAM Journal on Computing*, 35(4):855–881, 2006. doi: 10.1137/S0097539705447268. URL <https://www.cs.princeton.edu/courses/archive/spr04/cos598B/bib/BeierV.pdf>. Theorem 3 and Lemma 5 refer to the inspected conference version, Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC 2004).
- [4] René Beier, Heiko Röglin, Clemens Rösner, and Berthold Vöcking. The smoothed number of

- Pareto-optimal solutions in bicriteria integer optimization. *Mathematical Programming*, 200: 319–355, 2023. doi: 10.1007/s10107-022-01885-6. Published online September 27, 2022.
- [5] Aaresh Bhathena, Salar Fattahi, Andrés Gómez, and Simge Küçükyavuz. Solving convex quadratic optimization with indicators over structured graphs. arXiv:2603.02103v1, 2026. URL <https://arxiv.org/abs/2603.02103v1>.
 - [6] Aaresh Bhathena, Salar Fattahi, Andrés Gómez, and Simge Küçükyavuz. A parametric approach for solving convex quadratic optimization with indicators over trees. *Mathematical Programming*, 218:291–336, 2026. doi: 10.1007/s10107-025-02222-3. Published online in 2025; preprint arXiv:2404.08178v1, 2024.
 - [7] Daniel Bienstock and Tongtong Chen. Solving convex QPs with structured sparsity under indicator conditions. arXiv:2411.11722v1, 2024. URL <https://arxiv.org/abs/2411.11722v1>.
 - [8] Daniel Bienstock and Gonzalo Muñoz. LP formulations for polynomial optimization problems. *SIAM Journal on Optimization*, 28(2):1121–1150, 2018. doi: 10.1137/15M1054079.
 - [9] Hans L. Bodlaender. A linear-time algorithm for finding tree-decompositions of small treewidth. *SIAM Journal on Computing*, 25(6):1305–1317, 1996. doi: 10.1137/S0097539793251219.
 - [10] Tobias Brunsch and Heiko Röglin. Improved smoothed analysis of multiobjective optimization. *Journal of the ACM*, 62(1), 2015. doi: 10.1145/2699445. URL <https://arxiv.org/abs/1111.1546v2>. Inspected preprint: arXiv:1111.1546v2.
 - [11] Soobin Choi, Salar Fattahi, Shaoning Han, Andrés Gómez, and Leonardo Lozano. Convexification of mixed-integer quadratic optimization via decision diagrams. arXiv:2608.22815v1, 2026. URL <https://arxiv.org/abs/2608.22815v1>.
 - [12] Ali Çivril. A note on the hardness of sparse approximation. *Information Processing Letters*, 113(14–16):543–545, 2013. doi: 10.1016/j.ipl.2013.04.014. URL <https://www.alicivril.com/Complexity-SA.pdf>. Theorem locator refers to the author manuscript dated August 18, 2015.
 - [13] Abhimanyu Das and David Kempe. Algorithms for subset selection in linear regression. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pages 45–54, 2008. doi: 10.1145/1374376.1374384. URL <https://david-kempe.com/publications/regression.pdf>.
 - [14] Shaddin Dughmi and Tim Roughgarden. Black-box randomized reductions in algorithmic mechanism design. *SIAM Journal on Computing*, 43(1):312–336, 2014. doi: 10.1137/110843654. URL <https://theory.stanford.edu/~tim/papers/blackbox.pdf>.
 - [15] Hamza Fawzi and Pablo A. Parrilo. Exponential lower bounds on fixed-size psd rank and semidefinite extension complexity. arXiv:1311.2571v1, 2013. URL <https://arxiv.org/abs/1311.2571v1>.
 - [16] Samuel Fiorini, Serge Massar, Sebastian Pokutta, Hans Raj Tiwary, and Ronald de Wolf. Exponential lower bounds for polytopes in combinatorial optimization. *Journal of the ACM*, 62(2):Article 17, 2015. doi: 10.1145/2716307. Conference version: STOC 2012. Theorem locator refers to the inspected preprint dated March 16, 2015.

- [17] Stéphane Gaubert, William McEneaney, and Zheng Qu. Curse of dimensionality reduction in max-plus based approximation methods: Theoretical estimates and improved pruning algorithms. In *50th IEEE Conference on Decision and Control and European Control Conference*, pages 1054–1061, 2011. doi: 10.1109/CDC.2011.6161386. URL <https://arxiv.org/abs/1109.5241>.
- [18] Andrés Gómez, Shaoning Han, and Leonardo Lozano. Real-time solution of quadratic optimization problems with banded matrices and indicator variables. arXiv:2405.03051v1, 2024. URL <https://arxiv.org/abs/2405.03051v1>.
- [19] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.
- [20] László Kozma and Shay Moran. Shattering, graph orientations, and connectivity. *Electronic Journal of Combinatorics*, 20(3):P44, 2013. URL <https://www.combinatorics.org/ojs/index.php/eljc/article/download/v20i3p44/pdf>.
- [21] Muhamed Kuric, Jan Ahmetspahic, and Thomas Pock. Total generalized variation on a tree. *SIAM Journal on Imaging Sciences*, 17(2):1040–1077, 2024. doi: 10.1137/23M1556915.
- [22] Eugene L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972. doi: 10.1287/mnsc.18.7.401.
- [23] James R. Lee, Prasad Raghavendra, and David Steurer. Lower bounds on the size of semidefinite programming relaxations. In *Proceedings of the 47th Annual ACM Symposium on Theory of Computing*, pages 567–576, 2015. doi: 10.1145/2746539.2746599. URL <https://www.dsteurer.org/paper/sdpsize.pdf>. Theorem locators refer to the author manuscript, arXiv:1411.6317.
- [24] Jisun Lee, Andrés Gómez, and Alper Atamtürk. Convexification of multi-period quadratic programs with indicators. *Mathematical Programming*, 2026. doi: 10.1007/s10107-026-02379-5. Published online July 22, 2026.
- [25] Uri Lerner and Ronald Parr. Inference in hybrid networks: Theoretical limits and practical algorithms. In *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence*, pages 310–318, 2001. URL <https://arxiv.org/abs/1301.2288v1>. Inspected archive copy: arXiv:1301.2288v1.
- [26] Peijing Liu, Salar Fattahi, Andrés Gómez, and Simge Küçükyavuz. A graph-based decomposition method for convex quadratic optimization with indicators. *Mathematical Programming*, 2022. doi: 10.1007/s10107-022-01845-0. Preprint arXiv:2110.12547; inspected preprint dated October 26, 2021.
- [27] Peijing Liu, Alper Atamtürk, Andrés Gómez, and Simge Küçükyavuz. Polyhedral analysis of quadratic optimization problems with Stieltjes matrices and indicators. *Mathematical Programming*, 218:375–401, 2026. doi: 10.1007/s10107-025-02272-7. Published online in 2025.
- [28] Katta G. Murty. Computational complexity of parametric linear programming. *Mathematical Programming*, 19:213–219, 1980. doi: 10.1007/BF01581642.
- [29] Heiko Röglin and Shang-Hua Teng. Smoothed analysis of multiobjective optimization. In *Proceedings of the 50th Annual IEEE Symposium on Foundations of Computer Science*, pages

681–690, 2009. doi: 10.1109/FOCS.2009.21. URL <https://www.roeglin.org/publications/FOCS09.pdf>.

- [30] Linchuan Wei, Alper Atamtürk, Andrés Gómez, and Simge Küçükyavuz. On the convex hull of convex quadratic optimization problems with indicators. *Mathematical Programming*, 204: 703–737, 2024. doi: 10.1007/s10107-023-01982-0.